PROJECT EXPLAINER & INTERVIEW PREPARATION

ShrutiLipi

Paste a YouTube URL → get a transcript you can copy or download as `.txt`, `.srt` or `.vtt`. v1 is captions-only. A **pnpm-workspaces + Turborepo monorepo**: a Next.js 14 app on Vercel and a FastAPI API on Render, sharing one TypeScript contract package, plus a stdlib-only Python CLI — roughly **2,440 lines** of hand-written code across 33 files, with **4 pinned Python dependencies**, **3 JS runtime dependencies**, and Turbo orchestrating both halves from the root.

> **THE ONE-SENTENCE VERSION FOR AN INTERVIEW**
>
> ShrutiLipi turns a YouTube URL into a plain-text transcript. The interesting part isn't the transcript — it's that the naive way to get captions (asking YouTube directly) **fails from cloud hosts**, so the backend is built around an **environment-configurable provider fallback chain** that classifies each failure and reacts differently: fall through, narrow the remaining providers, or stop immediately.

FRONTEND
Next.js 14 App Router, TS, Tailwind
shrutilipi.nayankunwar.com

BACKEND
FastAPI + uvicorn, Python 3.11, Docker
shrutilipi-backend.onrender.com

API
`GET /api/transcript?url=&lang=`
`GET /api/wake` · `GET /health`

THIRD ENTRY POINT
`tools/fetch.py` — stdlib CLI
exit codes 0 / 1 / 2

KNOWN-GOOD TEST VIDEO
`HHUsHkYhkcM`
969 `en` segments

NAME RULE
display copy is always SHRUTILIPI

code slugs are `shrutilipi`

PART A

How the project works

Fifteen sections, from "what is it" down to the exact lines implementing the fallback chain. Every code block is real and copyable, and every `file:line` reference is accurate as of `main @ 774d74d` plus the uncommitted pnpm-workspaces/Turborepo migration (`frontend/ → apps/web`, `backend/ → apps/api`, new `packages/*`).

01 Overview

ShrutiLipi (*shruti* = hearing, *lipi* = script) is a YouTube transcript extractor. Three entry points, all going through the same backend.

What "v1 = captions-only" actually excludes

There is **no audio download and no speech-to-text**. If a video has no caption track the API returns `404 no_captions`. The frontend copy says so out loud: *"No captions found for this video. Try another video (Whisper fallback coming in v2)."*

This matters for interviews because it defines the seam for v2: add a `WhisperProvider` implementing the existing `TranscriptProvider` ABC, set `WHISPER_ENABLED=true`, append it to `CHAIN_ORDER`. **Zero frontend changes.** The abstraction was designed for the extension before the extension existed.

ENTRY POINT	FILE	TALKS TO	DEFINING PROPERTY
Web app	`apps/web/app/page.tsx`	Backend over CORS	The result URL *is* the share link
HTTP API	`apps/api/main.py`	— it *is* the API	2 routes, 1 cache, 1 error mapper
CLI	`tools/fetch.py`	Backend over HTTP	Stdlib only, scriptable exit codes

02 The constraint that shaped everything

Read this section before anything else. Almost every non-obvious decision in the repo traces back to this one fact.

THE FACT

YouTube blocks requests from datacenter IP ranges. On a home connection the obvious implementation works fine: call `youtube-transcript-api`, get captions, done. On Render's free tier it fails with `RequestBlocked` from `_assert_playability`, surfacing as a `502 youtube_blocked`.

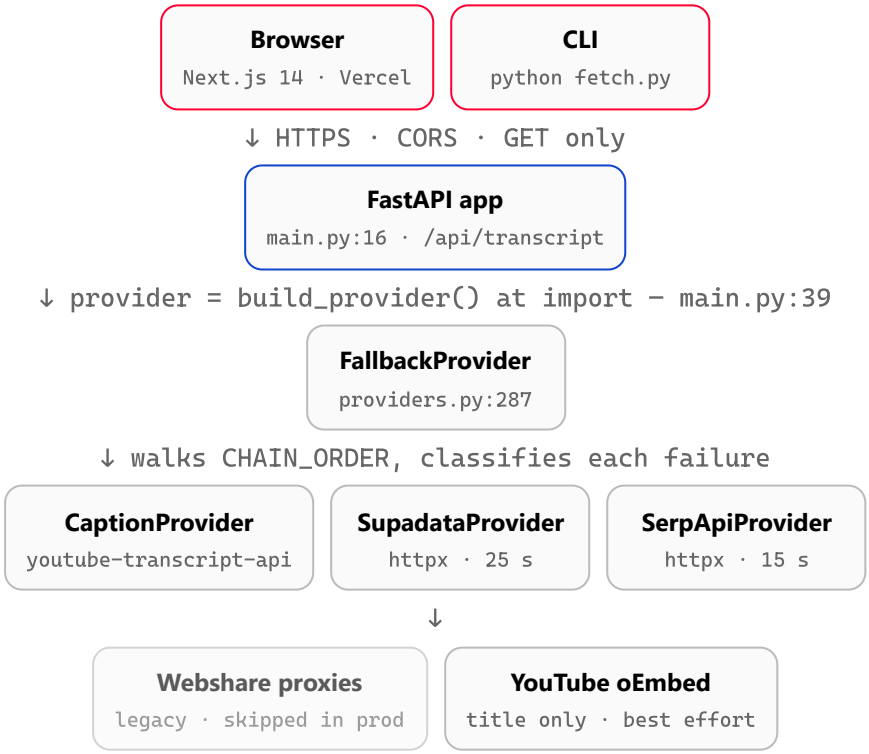
CONSEQUENCE	WHERE IT LIVES
Direct caption fetch can't be the only path in production	`DIRECT_ENABLED=false` in `render.yaml:11`
Need a hosted provider that transcribes from a different IP	`SupadataProvider` — `providers.py:112`
Need a last-resort provider too, for when the first is out of quota	`SerpApiProvider` — `providers.py:199`

CONSEQUENCE	WHERE IT LIVES
Need to choose between them at runtime without a redeploy per key	`build_provider()` — `providers.py:346`
Need a chain that understands *why* a provider failed	`FallbackProvider` — `providers.py:287`
Free Webshare proxies were the first fix; they're dead	`_get_proxy_config()` — `youtube.py:63`, kept as legacy and documented as “don't debug these”
Since the hosted path works, don't waste 10–20 s on a doomed proxy hop	`_hosted_levels_active()` short-circuit — `youtube.py:74`

HOW TO SAY THIS IN AN INTERVIEW

“The naive version of this app is thirty lines. It worked on my laptop and failed in production, because YouTube blocks datacenter IPs. That single failure is why the backend has an abstraction layer at all. Once you accept that transcript retrieval can fail in several *structurally different* ways — the video has no captions, your IP is blocked, your provider is out of quota, the video is age-restricted — you stop writing `try/except` and start writing a chain that knows which failures are worth retrying elsewhere and which ones *are* the answer.”

03 Architecture



Reading the diagram: the two clients are interchangeable. Everything below the FastAPI line is the part with the interesting design; everything above it is deliberately boring.

Design decisions worth naming

DECISION	RATIONALE	COST
Chain assembled at **import time**, not per request	Building it is cheap but pointless per request, and env is immutable at runtime anyway	Env changes need a redeploy — documented and deliberate

DECISION	RATIONALE	COST
Hosted providers use `httpx`, not vendor SDKs	Zero new dependencies; four Python deps total	Manual JSON-shape handling and a unit-guessing heuristic
No ORM, no database, no Pydantic models	Nothing is persisted; Pydantic would only restate the response	The response shape isn't self-documenting in OpenAPI
In-memory dict cache, not Redis	Single-instance free tier; captions for a given video barely change in 24 h	Doesn't survive restart, no shared cache, unbounded growth
No UI library — hand-inlined SVG icons	Three runtime JS deps; the brand colour is one Tailwind token	Icons aren't accessible out of the box
`plainText` derived, never stored	The copy output can never drift from `segments`	Recomputed per request, which is just a join

04 Repo map

Every hand-written file, what it's for, and how big it is. Total ≈ 2,834 lines.

FILE	LINES	RESPONSIBILITY
BACKEND — 1029 PYTHON LINES + WORKSPACE SHIM		
`apps/api/main.py`	149	App, CORS, transcript + wake routes, TTL cache, rate-limit gate, `_map_error`
`apps/api/services/ratelimit.py`	132	Per-IP token bucket (stdlib only), XFF client key, env factory, kill switch
`apps/api/services/providers.py`	369	**The centrepiece.** ABC, three providers, chain, error taxonomy, factory
`apps/api/services/youtube.py`	157	Video-ID regex ladder, oEmbed title, caption fetch, proxy config
`apps/api/tests/test_ratelimit.py`	199	16 pytest cases: bucket maths, key extraction, env parsing, TestClient 429 shape
`apps/api/tests/test_wake.py`	23	Wake-probe contract: `/api/wake` 200 shape + `/health` alias
`apps/api/Dockerfile`	12	`python:3.11-slim`, uvicorn on `${PORT:-8000}`
`apps/api/requirements.txt`	5	fastapi, uvicorn[standard], youtube-transcript-api, httpx, pytest — all pinned
`apps/api/package.json`	13	Thin Turbo shim — `dev` / `test` delegate to uvicorn / pytest; `build` is a `py_compile` syntax check
FRONTEND — 850 TS/TSX + 19 CSS + 56 CONFIG		
`apps/web/app/page.tsx`	159	Client state machine, bootstrap from query string, theme toggle, cold-start wake wiring
`apps/web/components/ShareButtons.tsx`	160	Five share targets plus copy link, no client SDKs
`apps/web/components/TranscriptView.tsx`	119	Render, timestamps toggle, copy, segmented download `.txt` / `.srt` / `.vtt`, share
`apps/web/components/ServerStatus.tsx`	126	Waking / awake / unreachable pill with retry — silent when the API is already warm
`apps/web/lib/useServerWake.ts`	101	Wake hook: ping on load, on tab return / first interaction, keep-warm while visible

FILE	LINES	RESPONSIBILITY
`apps/web/lib/api.ts`	49	`apiBase` + `pingHealth` (wake probe) + `fetchTranscript` — types and `friendlyError` live in shared
`apps/web/app/layout.tsx`	61	SEO metadata, anti-FOUC theme script (`SITE_URL` imported from shared)
`apps/web/components/UrlInput.tsx`	53	Input, language `<select>`, submit
`apps/web/app/globals.css`	19	Tailwind directives, `color-scheme` , `::selection`
`apps/web/app/sitemap.ts`	13	`/sitemap.xml` — one URL
`apps/web/app/robots.ts`	9	`/robots.txt` plus sitemap reference
`apps/web/package.json`	28	Now `@shrutilipi/web` ; depends on shared via `workspace:*` ; has a `typecheck` script
`apps/web/tsconfig.json`	11	Extends the shared base; keeps `jsx: preserve` + Next plugin
`apps/web/next.config.js`	17	`transpilePackages` + `outputFileTracingRoot` — both required for the monorepo build
SHARED PACKAGES + WORKSPACE ROOT — 322 LINES		
`packages/shared/src/index.ts`	85	**The contract.** Response types, `TranscriptErrorDetail` (incl. `rate_limited`), `friendlyError` , `formatTime` , `SITE_URL` ; re-exports `toSrt` / `toVtt`
`packages/shared/src/subtitles.ts`	46	SRT/VTT formatters (pure); two-language pair with `tools/fetch.py`
`packages/shared/src/subtitles.test.ts`	82	11 `node:test` cases pinning the edge-case contract (Node ≥22.6 strips TS natively)
`packages/shared/package.json`	22	`@shrutilipi/shared` ; ships raw TS, no build step; `node --test` script
`packages/shared/tsconfig.json`	11	Extends the shared base; `allowImportingTsExtensions` for the test's `.ts` import
`packages/tsconfig/base.json`	16	`@shrutilipi/tsconfig` — strict, bundler resolution, shared by web + shared
`packages/tsconfig/package.json`	6	Publishes `base.json` inside the workspace
`package.json`	21	Private root; pins `packageManager: pnpm` ; scripts delegate to Turbo
`pnpm-workspace.yaml`	3	Members: `apps/*` , `packages/*`
`turbo.json`	30	Pipelines: `build` (^build + `.next` outputs), persistent `dev` , `typecheck` , `test`
CLI, CONFIG & DOCS — 911 LINES		
`tools/fetch.py`	240	argparse CLI, single + batch, txt/srt/vtt formats, rich-optional output, exit codes
`tools/README.md`	171	Full CLI usage guide with dummy IDs only
`docs/operations.md`	219	**Best doc in the repo.** Env tables, chain semantics, deploy runbook, known issues with root causes
`docs/getting-started.md`	93	Run-it-yourself guide: prerequisites, ports, and the real troubleshooting table
`tools/BATCH.md`	89	Batch-mode deep dive

FILE	LINES	RESPONSIBILITY
`README.md`	73	Quickstart via `pnpm` + Turbo, deploy, v2 note
`render.yaml`	23	Render Blueprint: `rootDir: apps/api`, plan, eight env vars
`tools/shrutilipi.bat`	3	Windows launcher, `%~dp0` + exit-code passthrough

WHY PNPM WORKSPACES + TURBOREPO (AND WHAT IT IS NOT)

The migration moved `frontend/ → apps/web` and `backend/ → apps/api` with `git mv` (history preserved), deleted `package-lock.json` for a root `pnpm-lock.yaml`, and added `turbo.json` so `pnpm dev|build|typecheck` run from the root. The one piece of genuine shared value is `@shrutilipi/shared` — the API contract (types, `friendlyError`, `formatTime`, `SITE_URL`) in a single place, which immediately paid for itself by deleting the triplicated `SITE_URL`. The Python backend is not managed by pnpm; `apps/api/package.json` is a thin shim of delegating scripts so Turbo can start and check it. Nothing about the runtime behaviour changed — the deploy-relevant facts are that Vercel's Root Directory is now `apps/web` and Render's `rootDir` is now `apps/api`.

NOT PRESENT — AND THAT IS A TALKING POINT

No `.github/` (so no CI), no `pyproject.toml`, no ESLint or Prettier config, no `Makefile`. Tests have started: `apps/api/tests/test_ratelimit.py` pins the limiter and its 429 wiring (16 cases, `pnpm test`). The chain's hand-verified behaviours (chain parsing, `handles_missing` eligibility, terminal short-circuit, SerpApi time-unit inference, proxy skip) still need the same treatment — `operations.md:168` says “re-add as a test file if this grows.”

05 Data flow

The whole request, end to end, with the exact code that does each step.

```
[0] useServerWake.ts  GET /api/wake on page load -> starts the free-instance boot
 |  visible tab: re-ping every 10 min; the status pill explains any slow wake
 v
USER
 |  pastes URL  (or lands on /?url=<id>&lang=<code>)
 v
[1] page.tsx:61-72  bootstrap effect, guarded by the bootstrapped ref
 |  reads ?url= and ?lang=, setUrl(), auto-fetches ONCE
 v
[2] page.tsx:35  runFetch() -- re-entrancy guard, setLoading, clear error + data
 v
[3] lib/api.ts:27  fetchTranscript() -> GET {NEXT_PUBLIC_API_URL}/api/transcript
 |  cache: "no-store"   .   CORS preflight: GET, no custom headers
 v
[4] main.py:55  get_transcript()
 |  |-- youtube.py:27  extract_video_id()  4-regex ladder -> 400 invalid_url if None
 |  |-- main.py:65     cache hit on "videoId:lang" and age < 24h ? -> return immediately
 |  `-- cache miss -> ratelimit.py:92  token bucket -> 429 rate_limited if empty
 v
[5] providers.py:293  FallbackProvider.get_transcript()   <-- the interesting part
 |  for each level in CHAIN_ORDER:
 |     ok?                    -> return
 |     terminal?              -> re-raise NOW
 |     missing?               -> drop remaining levels with handles_missing = False
 |     blocked/quota/network? -> fall through
 v
[6] main.py:95  build payload  {videoId, title, language, plainText, segments[]}
 |  _CACHE["videoId:lang"] = (now, payload)   <-- only on success; failures never poison it
 v
[7] page.tsx:31  setData(result), then history.replaceState(?url=<canonical id>&lang=<lang>)
 v
[8] TranscriptView  plain text, or [m:ss] rows when timestamps are on
 |  Copy (clipboard -> textarea fallback) . Download .txt/.srt/.vtt (Blob) . Share (5 targets)
 v
CLI takes the same path from [3] onward, then: text -> stdout, stats -> stderr, exit 0/1/2
```

THE ELEGANT BIT

Step 7 rewrites the address bar to the *canonical resolved video ID*. So however the user arrived — a `youtu.be` short link with tracking junk, a raw 11-char ID, a full watch URL — the moment a result exists the URL becomes `/?url=dQw4w9WgXcQ&lang=en`. There is no result slug, no router, and no server render of results. **The current transcript result *is* the share link.** Share and Reload are the same code path.

06 Request lifecycle explorer

Step through the eight stages. Arrow keys work too.

All eight stages, in order

#	STAGE	WHERE	WHAT HAPPENS
1	**Land on the page — URL becomes state**	`apps/web/app/page.tsx:61–72`	• The app reads `?url=` and `?lang=` from the query string on mount. • A `bootstrapped` ref makes the effe fire exactly once, so a re-render can never trigger a second fetch (which would double-spend provider quota • If a URL is present it prefills the inpu and auto-fetches with the requested language. • A bare `/` with no params is a clean start state — no auto-fetch.
2	**Submit — re-entrancy guard**	`apps/web/app/page.tsx:35`	• `runFetch()` returns early if the inp blank *or* a request is already in flight • Loading is set, and the previous erro and result are both cleared so the U can't show a stale transcript next to new spinner. • The button is additionally `disabled={loading || !url.trim` — two independent guards against double spend on a 100-req/month f tier.
3	**HTTP request from the browser**	`apps/web/lib/api.ts:27`	• `apiBase()` reads `NEXT_PUBLIC_API_URL`, defaulting `http://localhost:8000` in code a stripping any trailing slash. • Params are `url` and `lang`, both `encodeURIComponent`-escaped. • `cache: "no-store"` — correctness beats a cached 404. • On a non-2xx response the body's `detail` is pulled out and run throu `friendlyError()`, which turns a machine code into a sentence.
4	**Validate the ID, then check the cache**	`apps/api/main.py:55–68`	• `extract_video_id()` tries four reg patterns in order: `watch?v=`, `youtu.be/`, `/shorts|embed|live|` then a bare 11-character ID. • If nothing matches, `400 invalid_u` immediately — no provider call is made. • The cache key is `"videoId:lang"`, different languages never collide. • A hit younger than 24 hours returns immediately. This is the single bigge reason the free-tier quota survives.

#	STAGE	WHERE	WHAT HAPPENS
5	**Walk the provider chain**	`apps/api/services/providers.py:293`	• `FallbackProvider` pops levels off mutable `pending` list and calls each turn. • Success (non-empty `segments`) ret immediately and logs which level served it. • An empty-but-successful result is its promoted to `TranscriptsNotFound` so "worked but found nothing" cour as a miss. • Any exception is caught, logged with the level name, and then classified.
6	**Classify the failure — three reactions**	`providers.py:280 / :268 / :321–324`	• **Terminal** (`VideoUnavailable`, `VideoUnplayable`, `AgeRestricted` → re-raise immediately. No provider can read a deleted or private video, retrying is guaranteed waste. • **Missing** (`NoTranscriptFound`, `TranscriptsDisabled`, …) → **narro** drop every remaining level with `handles_missing == False`. SerpA is captions-only, so it's a guaranteed miss that would spend quota. • **Blocked, quota, auth or network** - fall through. These are about *this* provider's reach, so a different provi on a different IP is genuinely likely t succeed. • On exhaustion the **last** error surface because it's the most recent and mo specific.
7	**Build the payload and cache it**	`apps/api/main.py:95–103`	• An empty `segments` list that somel reached here becomes `404 no_captions`. • `plainText` comes from the `plain_text` property, which joins `segments` — it is never stored separately, so it can't drift. • The cache is written **only on succes** transient provider outage therefore can't become a 24-hour one.

#	STAGE	WHERE	WHAT HAPPENS
8	**Render, and make the result shareable**	`apps/web/app/page.tsx:42, components/TranscriptView.tsx`	• On success the address bar is rewrit with `history.replaceState` to the **canonical resolved video ID**. • So a `youtu.be` short link, a full wat URL and a raw ID all converge on `/?url=<id>&lang=<code>`. The cur result *is* the share link. • `replaceState` rather than `pushSt` — the Back button shouldn't step through every search. • Then: copy with a `execCommand` fallback, download via `Blob`, or sha to five targets. • On failure `runFetch` catches and s message that is already human-readable, because `friendlyError` on the client

07 The provider chain

This is the part to be able to explain without looking. Everything else in the backend is bookkeeping.

LEVEL	SOURCE	HANDLES_MISSING	TIMEOUT	E
`CaptionProvider` `providers.py:68`	`youtube-transcript-api`, optional Webshare proxy	**True**	lib default	(
`SupadataProvider` `providers.py:112`	`GET api.supadata.ai/v1/transcript` + `x-api-key`. AI-retranscribes caption-less videos	**True**	25 s	+
`SerpApiProvider` `providers.py:199`	`GET serpapi.com/search?engine=youtube_video_transcript`. Captions only, no AI	**False**	15 s	+ (

THE ONE-LINE IDEA BEHIND HANDLES_MISSING

If a provider just told us *this video has no captions*, then calling another *captions-only* provider is guaranteed to fail and will burn quota. Supadata has AI transcription so it is worth trying; SerpApi is captions-only so it gets skipped. The chain narrows itself based on what it learned.

The data model

```
@dataclass
class TranscriptResult:
    video_id: str
    language: str
    segments: list = field(default_factory=list)
    title: str | None = None

    @property
    def plain_text(self) -> str:
        return " ".join(s["text"] for s in self.segments)
```

`plain_text` is a **property**, not a field. It is recomputed from `segments` every time, so the text the user copies can never disagree with the segments the user sees. Every provider returns this same shape, which is exactly what makes them interchangeable and the chain possible.

The interface and its policy switch

```
class TranscriptProvider(ABC):
    handles_missing: bool = True          # <- the policy switch that drives chain narrowing

    @abstractmethod
    def get_transcript(self, video_id: str, lang: str = "en") -> TranscriptResult:
        raise NotImplementedError
```

The factory

`build_provider()` is a three-way return, and all three branches matter:

```
def build_provider() -> TranscriptProvider:
    levels: list[TranscriptProvider] = []
    for name in _parse_chain_order():
        if name == "direct":
            if env_truthy(os.getenv("DIRECT_ENABLED"), True):
                levels.append(CaptionProvider())
        elif name == "supadata":
            key = os.getenv("SUPADATA_API_KEY", "").strip()
            if key and env_truthy(os.getenv("SUPADATA_ENABLED"), True):
                levels.append(SupadataProvider(key))
        elif name == "serpapi":
            key = os.getenv("SERPAPI_API_KEY", "").strip()
            if key and env_truthy(os.getenv("SERPAPI_ENABLED"), False):
                levels.append(SerpApiProvider(key))

    if not levels:
        logger.warning("chain empty (CHAIN_ORDER=%s) -- falling back to direct captions only",
                       ",".join(_parse_chain_order()))
        return CaptionProvider()      # (1) empty  -> bare direct. CORRECT for keyless local dev.
    if len(levels) == 1:
        return levels[0]              # (2) single -> return it unwrapped, no chain overhead
    return FallbackProvider(levels)   # (3) many   -> wrap in the chain
```

> **WHY BRANCH (2) EXISTS**
> With no API keys — the default local dev state — you get a bare `CaptionProvider`, not a one-element chain. So local development exercises the *real* direct path with zero indirection. Returning `FallbackProvider([CaptionProvider()])` would be functionally identical but would route every local request through the exception-handling machinery and log a warning per level, making local logs misleading.

`_parse_chain_order()` — silently drops junk

Config parsing is defensive on purpose. An unknown level name or a duplicate is **ignored**, not an error, so a typo in a Render env var degrades to "slightly different chain" instead of "service won't boot".

```
_CHAIN_DEFAULT = "direct,supadata,serpapi"
_KNOWN_LEVELS = ("direct", "supadata", "serpapi")


def _parse_chain_order() -> list:
    raw = os.getenv("CHAIN_ORDER", _CHAIN_DEFAULT)
    seen, order = set(), []
    for part in raw.split(","):
        name = part.strip().lower()
        if name in _KNOWN_LEVELS and name not in seen:
            seen.add(name)
            order.append(name)
    return order
```

Interview follow-up: `CHAIN_ORDER=serpapi,supadata,serpapi,BOGUS,serpapi` parses to `["serpapi", "supadata"]`. Order is preserved; duplicates and unknown names vanish. Ask why not raise on unknown — the answer is that a deploy-platform env var is a place where one typo takes down production, and a log warning doesn't.

`env_truthy()` — why the defaults differ per flag

```
def env_truthy(raw: Optional[str], default: bool = True) -> bool:
    """Parse env truthiness: 0/false/no/off -> False; empty/None -> default."""
    if raw is None or not raw.strip():
        return default
    return raw.strip().lower() not in ("0", "false", "no", "off")
```

Three states, not two: unset falls through to the provider's own default. `DIRECT_ENABLED` defaults `True` (you want captions to work out of the box on a home IP); `SERPAPI_ENABLED` defaults `False` (it costs money and is emergency-only). That asymmetry lives entirely in which boolean is passed as the second argument at the call site — easy to misread, which is why `render.yaml` sets `DIRECT_ENABLED=false` explicitly instead of relying on the default.

`SupadataProvider` — the production workhorse

```
resp = httpx.get(
    "https://api.supadata.ai/v1/transcript",
    params={"url": f"https://www.youtube.com/watch?v={video_id}", "lang": lang},
    headers={"x-api-key": self._api_key},
    timeout=self._timeout,                      # 25.0 s
)
_map_hosted_status(resp, "supadata")            # 401/403 auth, 402/429 quota, 404 not-found, 5xx network
if resp.status_code != 200:
    raise TranscriptsNotFound(f"supadata: unexpected {resp.status_code}")

data = _json_or_raise(resp, "supadata")         # non-JSON / non-dict -> ProviderNetworkError
if data.get("error"):
    _raise_hosted_error("supadata", data["error"])   # classify the error STRING

for entry in content:                           # content: [{text, offset(ms), duration(ms)}]
    segments.append({
        "start":    float(entry.get("offset")   or 0.0) / 1000.0,   # ms -> seconds
        "duration": float(entry.get("duration") or 0.0) / 1000.0,
        "text":     str(entry.get("text") or "").replace("\n", " ").strip(),
    })
```

Two response-shape defences: a malformed `content` list is tolerated (non-dict entries are skipped, empty text is dropped), and a completely empty result is promoted to `TranscriptsNotFound` rather than silently returning an empty transcript. Both

matter because "200 OK with garbage" is a real hosted-API failure mode.

`SerpApiProvider` and the time-unit inference heuristic

SerpApi's docs don't state whether `start` / `duration` are seconds or milliseconds. Guessing wrong turns a 3-minute talk into a 300-hour one. So the code **inspects the data**:

```
if unit in ("", "auto"):
    # seconds-based caption segments never reach 100s; ms-based rarely drop below 100ms
    unit = "ms" if (max(starts, default=0) > 50000 or max(durations, default=0) > 100) else "seconds"
divisor = 1000.0 if unit in ("ms", "milliseconds") else 1.0
```

Both signals are needed: a 10-minute video is 600 s, which is under 50 000, so the `max(start)` test alone would misfire — but its segment durations are around 3 s, which is over 100, so `max(duration)` catches it. Overridable via `SERPAPI_TIME_UNIT=seconds|ms` when the heuristic guesses wrong. A per-entry `TypeError` / `ValueError` is caught and coerced to `0.0`, so one bad row can't kill the batch.

Also in this method: if `transcripts` is absent it falls back to a plain `text` / `transcript` string as a single zero-timestamped segment, and the returned title comes from the same best-effort oEmbed call.

Caption fetching — the two-stage language ladder

`fetch_caption_transcript()` in `youtube.py:102` resolves language in two stages, and the fast path deliberately re-raises:

```
langs = [lang, "en"] if lang != "en" else ["en"]
# New instance per request: requests.Session is not thread-safe.
api = YouTubeTranscriptApi(proxy_config=_get_proxy_config())

# 1) Fast path
try:
    fetched = api.fetch(video_id, languages=langs)
    return _normalize(fetched.to_raw_data()), fetched.language_code
except Exception as first_exc:
    first_name = type(first_exc).__name__
    # Only fall through for "not found in requested langs".
    # Blocking / unavailable errors should propagate immediately.
    if first_name not in ("NoTranscriptFound", "TranslationLanguageNotAvailable"):
        raise

# 2) Fallback: list() ONCE, then pick the best available
transcript_list = api.list(video_id)
try:    t = transcript_list.find_transcript(langs)                      # exact + en
except: try: t = transcript_list.find_generated_transcript(langs)       # auto-generated
      except:
        try:    t = transcript_list.find_manually_created_transcript(all_codes)  # any manual
        except: t = next(iter(transcript_list))                         # literally anything
```

Preference order: **requested language → English → any manual → any auto-generated → whatever exists**. The `list()` result is reused across all four attempts, so the ladder costs one extra round-trip, not four.

> **WHY THE FAST PATH RE-RAISES**
>
> If the fast fetch fails because the IP is blocked or the video is gone, falling through to `list()` would issue a second doomed request. The guard restricts fall-through to exactly the two "wrong language" errors, so a block surfaces immediately instead of after a wasted round-trip.

08 The chain algorithm

`FallbackProvider.get_transcript()` — 36 lines that a linear `try/except` chain can't express.

```
def get_transcript(self, video_id: str, lang: str = "en") -> TranscriptResult:
    pending = list(self._levels)
    last_error: Exception | None = None

    while pending:
        level = pending.pop(0)
        name = type(level).__name__
        try:
            result = level.get_transcript(video_id, lang=lang)
            if result.segments:
                logger.info("transcript served by %s video_id=%s segments=%d",
                            name, video_id, len(result.segments))
                return result
            last_error = TranscriptsNotFound(f"{name} returned no segments")
        except Exception as exc:
            last_error = exc

        logger.warning("level %s failed video_id=%s: %s: %s",
                       name, video_id, type(last_error).__name__, last_error)
        if _is_terminal(last_error):        # (1) terminal  -> stop NOW
            raise last_error
        if _looks_missing(last_error):      # (2) missing   -> NARROW the survivors
            pending = [p for p in pending if p.handles_missing]

    if last_error is None:                 # (3) exhaustion -> surface the LAST error
        raise TranscriptError("chain exhausted without an error")
    raise last_error
```

FAILURE CLASS	DETECTED BY	REACTION	WHY THAT REACTION
Terminal `VideoUnavailable`, `VideoUnplayable`, `AgeRestricted`	`_is_terminal()` `providers.py:280`	**Stop immediately**, re-raise	No provider can read a private or deleted video. Retrying is guaranteed waste, and swallowing it hides a real answer from the user.
Missing `TranscriptsDisabled`, `NoTranscriptFound`, `NotTranslatable`, `TranslationLanguageNotAvailable`	`_looks_missing()` `providers.py:268`	**Narrow** pending to `handles_missing=True`	SerpApi is captions-only, so calling it after Supadata says “no captions” is a guaranteed miss that spends quota.
Blocked / quota / auth / network `IpBlocked`, `TooManyRequests`, `ProviderQuotaError`, `ProviderNetworkError`	falls through both checks	**Fall through** to the next level	These are about *this* provider's reach, not the video's content. A different provider on a different IP is genuinely likely to succeed.
Exhausted — no levels left	`while` exits with `last_error` set	Raise the **last** error	It's the most recent and most specific thing that happened; the first error is usually the least relevant.

Try it

The simulator in the next section replays this exact algorithm. Change the chain with the configurator, pick a failure scenario, and watch the loop choose between the three reactions.

09 Provider chain simulator

A faithful JavaScript port of `FallbackProvider.get_transcript()` (`providers.py:293–328`). It uses whatever chain the configurator in the next section produces, so change that first and come back.

Each scenario below replays the real `FallbackProvider` loop. The scenarios are written against the full `direct → supadata → serpapi` chain, so set the **Everything on** preset in the configurator first and then come back. The panel also shows the active chain and warns when a scenario needs a level your configuration doesn't have. Every scenario is traced in full in the printed table below.

Every scenario, traced step by step

Simulated against the full three-level chain `direct → supadata → serpapi`, because several scenarios depend on the `direct` level failing first. Re-run any of these against the deployed Render chain (`supadata → serpapi`) in the simulator above — two of the rows collapse, which is itself the point: with `DIRECT_ENABLED=false` there is no direct level to fail, so the "blocked" and "terminal" cases can no longer occur in production.

SCENARIO	TRACE	REACTION	OUTCOME
Happy path Direct captions work. This is what local development looks like.	1. **CaptionProvider** returned segments SERVED `logger.info("transcript served by CaptionProvider") → return result`	**served on first level**	**CaptionProvider** HTTP 200

Render IP blocked
The original production failure: `RequestBlocked` from the direct level, so it falls through to a hosted provider.
	1. **CaptionProvider** raised IpBlocked
	FAILED
	`logger.warning("level CaptionProvider failed: IpBlocked")`
	2. not missing, not terminal
	FALL THROUGH
	`fall through to the next level — the failure is about this provider's reach, not the video`
	3. **SupadataProvider** returned segments
	SERVED
	`logger.info("transcript served by SupadataProvider") → return result`
		fall through, then serves	**SupadataProvider**
			HTTP 200

SCENARIO	TRACE	REACTION	OUTCOME
No captions (AI saves it) Direct reports `TranscriptsDisabled`. The chain narrows to `handles_missing=True`, SerpApi is dropped, and Supadata's AI transcription succeeds.	1. **CaptionProvider** raised TranscriptsNotFound FAILED `logger.warning("level CaptionProvider failed: TranscriptsNotFound")` 2. _looks_missing() → True NARROW `pending = [p for p in pending if p.handles_missing]` — dropped **`serpapi`** (captions-only, would be a guaranteed miss) 3. **SupadataProvider** returned segments SERVED `logger.info("transcript served by SupadataProvider")` → return result	**fall through, then serves**	**SupadataProvider** HTTP 200

No captions anywhere
Every level reports missing. SerpApi is dropped by the narrowing, so the chain exhausts and surfaces Supadata's `TranscriptsNotFound` → 404.
	1. **CaptionProvider** raised TranscriptsNotFound FAILED
	`logger.warning("level CaptionProvider failed: TranscriptsNotFound")`
	2. _looks_missing() → True NARROW
	`pending = [p for p in pending if p.handles_missing]` — dropped **`serpapi`** (captions-only, would be a guaranteed miss)
	3. **SupadataProvider** raised TranscriptsNotFound FAILED
	`logger.warning("level SupadataProvider failed: TranscriptsNotFound")`
	4. _looks_missing() → True NARROW
	`pending = [p for p in pending if p.handles_missing]` — nothing left to drop
		narrow, then exhausted	HTTP **404**
			`no_captions`

SCENARIO	TRACE	REACTION	OUTCOME
Supadata out of quota A quota error is about Supadata's reach, not the video, so the chain falls through and SerpApi serves.	1. **CaptionProvider** raised IpBlocked FAILED logger.warning("level CaptionProvider failed: IpBlocked") 2. not missing, not terminal FALL THROUGH fall through to the next level — the failure is about this provider's reach, not the video 3. **SupadataProvider** raised ProviderQuotaError FAILED logger.warning("level SupadataProvider failed: ProviderQuotaError") 4. not missing, not terminal FALL THROUGH fall through to the next level — the failure is about this provider's reach, not the video 5. **SerpApiProvider** returned segments SERVED logger.info("transcript served by SerpApiProvider") → return result	**fall through, then serves**	**SerpApiProvider** HTTP 200
Video is private Nothing can read a private video, so the chain stops on the first terminal error and never calls a paid provider.	1. **CaptionProvider** raised VideoUnavailable FAILED logger.warning("level CaptionProvider failed: VideoUnavailable") 2. _is_terminal() → True STOP raise immediately — the chain stops, no further provider is called or billed	**short-circuit**	HTTP **404** `video_unavailable`

SCENARIO	TRACE	REACTION	OUTCOME

All providers fail
Blocked, then quota. No levels left, so the *last* error surfaces → 502 `provider_unavailable`.
	1. **CaptionProvider** raised IpBlocked FAILED
	`logger.warning("level CaptionProvider failed: IpBlocked")`
	2. not missing, not terminal FALL THROUGH
	`fall through to the next level — the failure is about this provider's reach, not the video`
	3. **SupadataProvider** raised ProviderQuotaError FAILED
	`logger.warning("level SupadataProvider failed: ProviderQuotaError")`
	4. not missing, not terminal FALL THROUGH
	`fall through to the next level — the failure is about this provider's reach, not the video`
	5. **SerpApiProvider** raised ProviderQuotaError FAILED
	`logger.warning("level SerpApiProvider failed: ProviderQuotaError")`
	6. not missing, not terminal FALL THROUGH
	`fall through to the next level — the failure is about this provider's reach, not the video`
		fall through, exhausted	HTTP **502**
			`provider_unavailable`

Read the last three rows together. “No captions (AI saves it)” and “No captions anywhere” take the *identical* reaction — narrow away the captions-only provider — and differ only in whether Supadata succeeded. That is the whole argument for `handles_missing`: the reaction is driven by what was learned about the video, not by which provider happened to be configured. And “Video is private” stops on step one, so neither paid provider is ever called.

10 Environment-variable configurator

Toggles for the exact conditions in `build_provider()` and `_parse_chain_order()`. On screen the resulting chain, what the factory returns and what the proxy resolver does are shown live. The printed table below covers the same logic for six named situations.

What each environment produces

SITUATION	CHAIN_ORDER	RESULTING CHAIN	BUILD_PROVIDER() RETUR
Local dev, no keys at all	`direct,supadata,serpapi`	direct	`CaptionProvider [unwrapped`

SITUATION	CHAIN_ORDER	RESULTING CHAIN	BUILD_PROVIDER() RETUR
Render production (as deployed)	`supadata,serpapi`	supadata	`SupadataProvider [unwrappe`
SerpApi key set but flag left false	`supadata,serpapi`	supadata	`SupadataProvider [unwrappe`
Full chain, all three active	`direct,supadata,serpapi`	direct→supadata→serpapi	`FallbackProvider([3 levels`
Typo and duplicates in CHAIN_ORDER	`serpapi,supadata,serpapi,BOGUS,serpapi`	serpapi→supadata	`FallbackProvider([2 levels`
Everything disabled	`direct,supadata,serpapi`	serpapi	`SerpApiProvider [unwrapped`

Row 3 is the documented gotcha: a SerpApi key with `SERPAPI_ENABLED` unset is silently ignored, because that flag defaults to false while `DIRECT_ENABLED` and `SUPADATA_ENABLED` default to true. Row 5 shows `_parse_chain_order()` dropping duplicates and the unknown name instead of raising. Row 6 is the "chain empty" warning path, which returns a bare direct provider rather than failing to boot.

11 Error taxonomy

There are two classification layers, on purpose. Knowing *why* both exist is the interesting part.

Layer 1 — typed exceptions carrying a `detail`

Everything the new code raises is one of five classes, each carrying the `detail` string that becomes the API's `detail` field:

```
class TranscriptError(Exception):
    detail = "transcript_fetch_failed"

class TranscriptsNotFound(TranscriptError):   detail = "no_captions"
class ProviderQuotaError(TranscriptError):     detail = "provider_unavailable"
class ProviderAuthError(TranscriptError):      detail = "provider_unavailable"
class ProviderNetworkError(TranscriptError):   detail = "provider_unavailable"
```

Quota, auth and network collapse to one `detail` deliberately: the user cannot act differently on any of them ("retry in a few minutes"), and from outside, distinguishing them would leak which provider is in use and whether your key is still valid.

Layer 2 — class-name sets plus message sniffing

Exceptions from the third-party `youtube-transcript-api` are **not wrapped**, so they carry no `detail`. The code classifies them by `type(exc).__name__` against three name sets, with substring fallbacks:

```
_BLOCK_NAMES   = {"IpBlocked", "RequestBlocked", "FailedToCreateConsentCookie",
                  "PoTokenRequired", "YouTubeRequestFailed", "YouTubeDataUnparsable",
                  "TooManyRequests"}
_MISSING_NAMES = {"TranscriptsDisabled", "NoTranscriptFound", "NotTranslatable",
                  "TranslationLanguageNotAvailable"}
_TERMINAL_NAMES = {"VideoUnavailable", "VideoUnplayable", "AgeRestricted"}
```

THE GENUINELY TRICKY CASE

`CouldNotRetrieveTranscript` is a catch-all in the library: it wraps IP blocks, consent-cookie failures *and* genuinely-missing transcripts. It appears in *no* name set, so both `_looks_missing()` and `_map_error()` fall back to reading the message — and the two read it differently on purpose:

- In `providers.py:273–276` the **block** check runs **first**: `"blocking requests"` or (`"ip"` and `"block"`) returns `False` for *missing*. A wrapped IP block is therefore never mistaken for "no captions", which would wrongly narrow the chain and return a 404.
- In `main.py:99–108`, `"could not retrieve"` *is* a missing signal, but lines 106–107 check for the IP-block phrasing first and return `502 youtube_blocked` instead of `404 no_captions`.

Both do the specific check before the generic one. Say this out loud in an interview — it demonstrates you've actually met the failure modes rather than pattern-matched a tutorial.

Full mapping: exception → HTTP → `detail` → user-facing copy

CONDITION	HTTP	`DETAIL`	WHAT THE USER SEES (`SHARED/SRC/INDEX.TS:53`)
Unparseable URL or ID	400	`invalid_url`	"That doesn't look like a valid YouTube URL or video ID."
No caption track anywhere	404	`no_captions`	"No captions found for this video. Try another video (Whisper fallback coming in v2)."
Deleted, private or age-restricted	404	`video_unavailable`	"Video is unavailable, private, or age-restricted."
YouTube blocked us	502	`youtube_blocked`	"YouTube temporarily blocked the request. Wait a minute and retry."
Quota, auth or network, chain exhausted	502	`provider_unavailable`	"Transcript services are busy or out of quota — retry in a few minutes."
Too many requests (token bucket, cache misses only)	429	`rate_limited`	"You're requesting transcripts a bit too quickly — wait a minute and retry." Plus a `Retry-After` header. Keyed by leftmost `X-Forwarded-For` (Render proxies); 20 req / 10 min per client via `RATE_LIMIT_*` env.
Anything unrecognised	502	`transcript_fetch_failed`	"Couldn't fetch the transcript. Please retry."

WHY 502 AND NOT 500 FOR A BLOCKED YOUTUBE

ShrutiLipi is up; the upstream it depends on refused. 502 Bad Gateway is the honest code, and it means something specific to monitoring and to retry policies. 500 would claim "we have a bug", which is both untrue and unactionable. A 404 for "no captions" is equally honest: the resource genuinely doesn't exist.

`_map_error()` in full — the exact precedence order

```
def _map_error(exc: Exception) -> HTTPException:
    if isinstance(exc, TranscriptError):                    # (0) typed errors win outright
        return HTTPException(
            status_code=404 if exc.detail == "no_captions" else 502,
            detail=exc.detail,
        )

    name = type(exc).__name__
    msg = str(exc).lower()

    if name in ("InvalidVideoId",) or ("invalid" in msg and "video" in msg):
        return HTTPException(400, detail="invalid_url")
    if name in ("VideoUnavailable", "VideoUnplayable", "AgeRestricted") or "video unavailable" in msg:
        return HTTPException(404, detail="video_unavailable")
    if name in ("TranscriptsDisabled", "NoTranscriptFound", "NotTranslatable",
                "TranslationLanguageNotAvailable") \
       or "no transcript" in msg or "transcripts disabled" in msg or "could not retrieve" in msg:
        # CouldNotRetrieveTranscript wraps many causes; distinguish blocking by message
        if "blocking requests from your ip" in msg or "ip" in msg and "block" in msg:
            return HTTPException(502, detail="youtube_blocked")
        return HTTPException(404, detail="no_captions")
    if name in ("IpBlocked", "RequestBlocked", "FailedToCreateConsentCookie",
                "PoTokenRequired", "YouTubeRequestFailed", "YouTubeDataUnparsable",
                "TooManyRequests") or ("ip" in msg and "block" in msg):
        return HTTPException(502, detail="youtube_blocked")
    return HTTPException(502, detail="transcript_fetch_failed")
```

Read it as an ordered list of guards where the first match wins. The typed-error branch is first, so a deliberate `ProviderQuotaError` can never be reclassified by a lucky substring match. The substring fallbacks such as `"invalid" in msg and "video" in msg` are a hedge against upstream exception classes being renamed: a rename then degrades to a generic 502 instead of an unhandled 500.

Classifying a hosted provider's *error string*

Hosted APIs return HTTP 200 with an `error` field in the body, so status-code mapping alone isn't enough. Two helpers handle it:

```
def _raise_hosted_error(source: str, msg: str) -> None:
    low = str(msg).lower()
    if any(h in low for h in ("quota", "credit", "exhaust", "rate limit", "too many",
                              "billing", "plan")):
        raise ProviderQuotaError(f"{source}: {msg}")
    if any(h in low for h in ("api key", "api_key", "unauthorized", "forbidden", "invalid key")):
        raise ProviderAuthError(f"{source}: {msg}")
    raise ProviderNetworkError(f"{source}: {msg}")


def _map_hosted_status(resp, source: str) -> None:
    if resp.status_code in (401, 403):  raise ProviderAuthError(f"{source}: auth failed ({resp.status_code})")
    if resp.status_code in (402, 429):  raise ProviderQuotaError(f"{source}: quota/rate limited ({resp.status_co
    if resp.status_code == 404:         raise TranscriptsNotFound(f"{source}: transcript not found")
    if resp.status_code >= 500:       raise ProviderNetworkError(f"{source}: upstream {resp.status_code}")
```

The keyword lists are informed guesswork about third-party wording, and that's an honest weakness worth naming: if Supadata changes its error text, a quota error silently becomes a network error. Both map to the same HTTP 502 and the same user-facing string, so the blast radius is small — which is exactly why quota, auth and network share a `detail`. `402` is included because that's how both vendors signal “out of credit”.

12 Cache, CORS, environment

The cache

```
_CACHE: dict[str, tuple[float, dict]] = {}
CACHE_TTL = 60 * 60 * 24          # 24 hours

cache_key = f"{video_id}:{lang}"
hit = _CACHE.get(cache_key)
if hit and (time.time() - hit[0]) < CACHE_TTL:
    return hit[1]
# ... on success only:
_CACHE[cache_key] = (time.time(), payload)
```

PROPERTY	VALUE	CONSEQUENCE — WOULD YOU CHANGE IT?
Key	`"videoId:lang"`	Includes lang, so `en` and `hi` are separate entries. Correct.
TTL	24 h	Captions for a given video effectively don't change, and it cuts repeat calls against a 100-req/month free tier.
Eviction	**None**	No LRU, no size cap. Unbounded growth, so a long-lived process leaks memory slowly. First thing I'd fix: a max size, or a periodic sweep of expired keys.
Scope	Process-local	Lost on restart and not shared between instances. Fine on Render's free single instance; breaks the moment you run 2+ workers, since each holds its own cache and you pay twice for popular videos.
Writes	Success only	Failures never poison the cache — important, because a poisoned entry would turn a transient provider outage into a 24-hour one.
Locking	None	Under the GIL, dict writes are atomic enough here. But 20 concurrent misses on the same cold video all hit the provider — a **thundering herd**. Would add per-key single-flight locking or a short negative cache.

CORS

```
PROD_ORIGINS = (
    "https://shrutilipi.nayankunwar.com",
    "https://www.shrutilipi.nayankunwar.com",
    "https://shrutilipi-rho.vercel.app",
)

def _allowed_origins() -> set[str]:
    origins = set(PROD_ORIGINS) | set(LOCAL_ORIGINS)
    for part in os.getenv("FRONTEND_URL", "").split(","):
        part = part.strip()
        if part:
            origins.add(part)
    return origins

_allowed = _allowed_origins()

app.add_middleware(
    CORSMiddleware,
    allow_origins=sorted(_allowed),     # a set, sorted for deterministic output
    allow_origin_regex=r"^http://(localhost|127\.0\.0\.1):\d+$",  # any local port
    allow_credentials=False,             # no cookies -> credentials not needed
    allow_methods=["GET", "OPTIONS"],
    allow_headers=["*"],
)
```

- **Allowlist, not wildcard.** `allow_origins=["*"]` would be simpler and is fine for a public read-only API, but an explicit list means a compromised or misconfigured frontend var can't open the API to everything.

- **Localhost by regex.** `allow_origin_regex` matches any `localhost` or `127.0.0.1` port, because `next dev` drifts ports when 3000 is taken — a fixed 3000-only list broke local dev the day a second project claimed the port (server logs 200, browser says *CORS error*). Production origins stay explicit; the regex can never match a real deployment.
- **Read-only surface.** `["GET", "OPTIONS"]` — there is no mutating endpoint, so the CORS policy can be maximally tight. Good interview observation: CORS is a *browser* mechanism, not access control. It stops other *web pages* from reading your responses inside a user's browser; it does nothing against `curl`. The CLI proves it — it's a non-browser client and works fine.
- `allow_credentials=False` because there is no auth at all in v1. Note this also means `Access-Control-Allow-Origin: *` would be legal here, which is exactly the check to make the day you *do* add auth.
- **Comma-separated multi-origin** supports preview deployments. The real cost: `FRONTEND_URL` must match the frontend domain *exactly*, scheme included. A mismatch appears as a browser CORS error with no server-side log — which cost one deploy to learn, and is why `docs/operations.md` calls it out.
- **Production origins are hardcoded, not just env.** A custom domain was added to the Vercel project while the API's allowlist still held only the Vercel URL, so the browser sent an `Origin` the API did not know: preflight returned `400 "Disallowed CORS origin"` and the simple `GET /api/wake` came back 200 *without* `access-control-allow-origin`, so the browser discarded every response. The page loaded perfectly; no API call worked. `FRONTEND_URL` is now additive-only — it can add origins but can no longer lock the site out of its own API.
- **Split before adding.** The old code added the raw env string to the set *before* splitting it, so `"https://a.app,https://b.app"` landed in the allowlist as one bogus origin (the two real ones were added too, so it mostly worked by accident). Now it is split first — and the regression is pinned by a test.

All seventeen environment variables

VARIABLE	DEFAULT	ON RENDER	READ AT	PURPOSE
`FRONTEND_URL`	`http://localhost:3000`	comma-separated origins	`main.py:40`	CORS allow-origin; additive only — pro origins are hardcode
`PORT`	`8000`	platform-set	`Dockerfile`	uvicorn bind port via `${PORT:-8000}`
`CHAIN_ORDER`	`direct,supadata,serpapi`	`supadata,serpapi`	`providers.py:336`	Try-order; unknown names and duplicate dropped
`DIRECT_ENABLED`	**`true`**	**`false`**	`providers.py:350`	Render's IP is YouTube-blocked
`SUPADATA_API_KEY`	—	**secret, required**	`providers.py:353`	Supadata dashboard key
`SUPADATA_ENABLED`	`true`	unset	`providers.py:354`	Kill switch without deleting the key
`SERPAPI_API_KEY`	—	secret, optional	`providers.py:357`	Backup provider key
`SERPAPI_ENABLED`	**`false`**	**not in render.yaml**	`providers.py:358`	A key alone does nothing without this flag
`SERPAPI_TIME_UNIT`	`auto`	unset	`providers.py:168`	`auto` | `seconds` | `ms` — overrides the heuristic
`WEBSHARE_PROXY_LIST`	—	—	`youtube.py:79`	Legacy: comma-separated full proxy URLs
`WEBSHARE_PROXY_USERNAME`	—	secret, leave empty	`youtube.py:86`	Legacy proxy credentials

VARIABLE	DEFAULT	ON RENDER	READ AT	PURPOSE
`WEBSHARE_PROXY_PASSWORD`	—	secret, leave empty	`youtube.py:87`	Legacy proxy credentials
`WEBSHARE_PROXY_HOSTS`	—	secret, leave empty	`youtube.py:88`	Legacy `host:port` list
`RATE_LIMIT_ENABLED`	`true`	unset	`ratelimit.py:120`	Kill switch: `0` / `false` / `no` / `of` disables the limiter
`RATE_LIMIT_REQUESTS`	`20`	unset	`ratelimit.py:124`	Bucket capacity per client per window
`RATE_LIMIT_WINDOW_SECONDS`	`600`	unset	`ratelimit.py:125`	Refill window in seconds; misses only `/api/wake` unlimited
`NEXT_PUBLIC_API_URL`	`http://localhost:8000`	Vercel env	`lib/api.ts:3`	Browser-visible API base

All keys are `sync: false` in `render.yaml`, so they must be typed into Render → Environment → Secrets by hand and **no secret is ever in git**. Editing an env var on Render requires **Save, rebuild, and deploy**, because `build_provider()` reads env at import time.

13 Frontend

Next.js 14 App Router, TypeScript in strict mode, Tailwind with a single brand token. No UI library, no state library, no icon package.

State: five `useState` and one `useRef`

```
const [url, setUrl] = useState("");
const [lang, setLang] = useState("en");
const [loading, setLoading] = useState(false);
const [error, setError] = useState<string | null>(null);
const [data, setData] = useState<TranscriptResponse | null>(null);
const [dark, setDark] = useState(true);
const bootstrapped = useRef(false);        // <- a REF, not state. See below.
```

Rendering is a flat conditional with no state-machine library: `<ServerStatus>` (invisible unless the free instance is waking or unreachable) and `<UrlInput>` always, then exactly one of the loading card, the error card, or `<TranscriptView>`. The error card gained a retry button, and the loading card switches its copy after 3 s to name the cold-start possibility instead of pretending to be fast.

The once-only bootstrap

```
useEffect(() => {
  if (bootstrapped.current) return;      // fire exactly once
  bootstrapped.current = true;
  const presetLang = searchParams.get("lang");
  const presetUrl  = searchParams.get("url");
  if (presetLang) setLang(presetLang);
  if (presetUrl) {
    setUrl(presetUrl);
    void runFetch(presetUrl, presetLang || "en");
  }
}, [searchParams]);
```

WHY A REF AND NOT A STATE FLAG

The effect depends on `searchParams`, whose object identity can change on re-render. A `useState` flag would be read as stale within the same effect pass and, worse, setting it would schedule an extra render — the classic "effect runs twice because I set state inside it" bug. A `useRef` mutation is not a render trigger, so the guard is free. It also survives `React.StrictMode`'s double-invoke in dev, which matters because `next.config.js` sets `reactStrictMode: true` — a double fetch would burn provider quota.

Re-entrancy guard — protecting paid quota

```
async function runFetch(urlValue: string, langValue: string) {
  if (!urlValue.trim() || loading) return;      // guard 1: in code
  setLoading(true); setError(null); setData(null);
  try {
    const result = await fetchTranscript(urlValue.trim(), langValue);
    setData(result);
    try {
      const qs = `?url=${encodeURIComponent(result.videoId)}&lang=${encodeURIComponent(langValue)}`;
      window.history.replaceState(null, "", qs);
    } catch {
      // history API unavailable -- shareable link just won't update
    }
  } catch (e) {
    setError(e instanceof Error ? e.message : "Something went wrong.");
  } finally {
    setLoading(false);
  }
}
```

Guard 1 is the early return; guard 2 is `disabled={loading || !url.trim()}` on the button in `UrlInput.tsx`. Belt and braces against a double spend on a 100-req/month free tier.

▾ **Progressive-enhancement details worth naming**

- **Clipboard with a fallback.** `navigator.clipboard.writeText` is unavailable in non-secure contexts, so both `TranscriptView.tsx:16` and `ShareButtons.tsx:99` fall back to a hidden `<textarea>` plus `document.execCommand("copy")`.
- **Theme before first paint.** `layout.tsx:41` injects a raw IIFE into `<head>` that reads `localStorage["shrutilipi-theme"]` and adds the `dark` class before React hydrates, with `suppressHydrationWarning` on `<html>` because the server always renders the light-less markup. Without it, dark-mode users get a white flash on every load.
- **Download without a library.** `Blob` + `URL.createObjectURL` + `revokeObjectURL`, one button group for `.txt` / `.srt` / `.vtt` with filename `${videoId}.<fmt>` (`TranscriptView.tsx`). The subtitle files come from pure `toSrt` / `toVtt` functions in `packages/shared/src/subtitles.ts` — no library, and the same edge-case rules are mirrored in the Python CLI's twins.
- **Share without SDKs.** `ShareButtons.tsx` hand-builds `wa.me/?text=`, `t.me/share/url`, the X intent, the Facebook sharer and `mailto:` from a single `${origin}/?url=<id>&lang=<lang>` string. Every external link gets `rel="noopener noreferrer"`.
- **Defensive browser calls.** `history.replaceState` and `localStorage.setItem` both sit in `try/catch` with a comment explaining the graceful degradation, instead of a bare `catch {}`.
- `cache: "no-store"` in `fetchTranscript` — correctness beats a cached 404.

▾ `formatTime()` — **implemented twice, once per layer**

`packages/shared/src/index.ts:73` (TypeScript, imported by `TranscriptView`) and `tools/fetch.py:42` (Python) implement the same rule: if hours are greater than zero print `h:mm:ss`, otherwise `m:ss`. A 5-minute video must not read `0:05:30`.

```
function formatTime(s: number): string {
  const total = Math.max(0, Math.floor(s));
  const h = Math.floor(total / 3600);
  const m = Math.floor((total % 3600) / 60);
  const sec = total % 60;
  if (h > 0) return `${h}:${String(m).padStart(2, "0")}:${String(sec).padStart(2, "0")}`;
  return `${m}:${String(sec).padStart(2, "0")}`;
}
```

Duplication across a network boundary is acceptable — sharing it would require a shared runtime or server-rendered formatting. Flag it as a known trade-off rather than pretending it doesn't exist.

14 The CLI

`tools/fetch.py` — 184 lines, **zero required dependencies**. `rich` is imported inside `try/except ImportError`, so it enhances output when present and is completely optional.

```
python tools/fetch.py HHUsHkYhkcM
python tools/fetch.py https://youtu.be/HHUsHkYhkcM --lang hi --timestamps
python tools/fetch.py HHUsHkYhkcM --out transcript.txt
python tools/fetch.py HHUsHkYhkcM --srt --out subs.srt   # SubRip cues (or --vtt)
python tools/fetch.py HHUsHkYhkcM --out -            # writes HHUsHkYhkcM.txt
python tools/fetch.py --batch urls.txt --plain       # one URL per line, # comments
python tools/fetch.py X --api https://shrutilipi-backend.onrender.com

python tools/fetch.py HHUsHkYhkcM | pbcopy            # transcript only, no stats
```

FLAG	MEANING
`url` **POSITIONAL, OPTIONAL**	YouTube URL in any of four shapes, or a raw 11-char ID
`--lang`	Caption language, default `en`
`--api`	Backend base URL, default `http://localhost:8000`
`--srt` / `--vtt`	Export SubRip or WebVTT cues instead of text (mutually exclusive; `--timestamps` no longer applies)
`--out`	Write to a file instead of stdout; `-` means `<videoId>.txt`
`--timestamps`	Prefix each line with `[m:ss]`
`--batch <file>`	One URL per line; `#` comments and blank lines are skipped
`--plain`	Force plain output even when `rich` is installed

The two decisions that matter

1. stdout versus stderr discipline

The transcript goes to **stdout**; every statistic, progress line and error goes to **stderr** (`print(msg, file=sys.stderr)`). That's what makes `python tools/fetch.py ID | pbcopy` and `> notes.txt` produce clean output. `rich.Console(stderr=True)` is constructed with `stderr=True` for the same reason. Pure Unix discipline, in a Python script.

2. Exit codes as an API

```
except urllib.error.HTTPError as e:
    try:
        detail = json.load(e).get("detail", "")
    except Exception:
        detail = ""
    fail_msg(url, f"HTTP {e.code} {detail}", pretty)
    return 1 if detail in ("no_captions", "invalid_url") else 2
except Exception as e:
    fail_msg(url, str(e), pretty)
    return 2
```

`0` ok · `1` no captions or bad URL — *expected, not a failure of the tool* · `2` network or server error. The CLI's exit code is derived from the backend's `detail` string, so the taxonomy is a genuine contract across both layers. In batch mode the returned code is `worst = max(worst, rc)`, so `0` genuinely means every URL landed.

▾ Batch mode and the Windows launcher

Batch (`run_batch` , `fetch.py:110`) is sequential, and each fetch is forced to `out="-"` so every video writes `<videoId>.txt` in the current directory. A failure does **not** abort the run — it records a row and continues. With `rich` you get a transient `Progress` bar (Spinner / Text / Bar / MofN / TimeElapsed) and a green-red summary `Table` ; without it, `[i/n] <url>` lines on stderr plus a `batch done: ok/total ok (exit N)` footer. Sequential rather than parallel on purpose: it keeps provider quota predictable and output ordering stable.

Launcher — `tools/shrutilipi.bat` , annotated in `tools/README.md:72` :

```
@echo off                          REM silence cmd's own command echo
python "%~dp0fetch.py" %*          REM run fetch.py next to this .bat, pass all args through
exit /b %errorlevel%               REM propagate exit code (0/1/2) back to cmd
```

`%~dp0` is the script's own folder, which makes it cwd-independent and space-safe. `%*` passes every argument through unchanged, and the explicit `exit /b %errorlevel%` keeps the exit-code contract intact through the batch layer. Add the folder to `PATH` once and you can run `shrutilipi <id>` from anywhere.

> **FAIR-USE NOTE TO REPEAT IF ASKED**
> "The CLI hits the same free-tier quota as the web app — 100 requests a month on Supadata's free tier. Batch files should stay small, and the CLI respects the same rate limiter as the browser: hit it too fast and you get a `429 rate_limited` with a `Retry-After` header, so automation should back off rather than retry blindly."

15 Known gaps & trade-offs

Six things I'd still fix, with the reasoning already formed — plus four already improved and marked: three fixed in code (the monorepo migration fixed two, rate limiting the third) and cold starts now mitigated by

a wake-up UI. Being able to criticise your own work is worth more in an interview than having nothing to criticise.

GAP	WHY IT EXISTS	HOW I'D FIX IT
No tests for the chain	The limiter has 16 pytest cases, but the chain has none: five behaviours were hand-verified and `operations.md:168` admits it. The chain logic is pure functions of (chain, outcome) — the easiest possible test target.	`pytest` against `build_provider()` with `monkeypatch.setenv` (five table-driven cases), `FallbackProvider` with stub levels (six failure paths), `_normalize_serpapi` units, and `extract_video_id` for all four URL shapes. A few hours for a large confidence gain.
No CI	Solo project; deploy is push-to-Render.	GitHub Actions running `pytest` plus `pnpm build` and `pnpm typecheck` (both already exist as Turbo pipelines). Note there is still no `lint` script — the repo has no ESLint config yet.
Unbounded cache	Shipped for the single-instance free tier, where no eviction was needed yet.	Cap the size (say 500 entries, drop oldest) plus a periodic sweep of expired keys. If it ever runs multi-worker, move to Redis so the cache is shared.
Thundering herd on cold keys	No per-key locking, so 20 concurrent requests for the same new video all hit the provider.	Per-key `asyncio.Lock` or single-flight so the first request populates the cache and the rest await it. A short negative cache for misses would also protect quota.
Rate limiting FIXED **, no auth**	v1 is a public free tool; auth adds friction and there's no user model.	Rate limiting shipped: per-IP token bucket in `services/ratelimit.py` (misses only, 20 req / 10 min, `429 rate_limited` + `Retry-After`), with 16 pytest tests. Auth remains the bigger unbuilt change: user records, a database, and secret management.
Cold starts on the free instance MITIGATED	Render's free tier sleeps after ~15 min idle; the next request pays ~30 s. Nothing in the UI said so — it just looked broken.	Wake-up UX shipped: `pingHealth()` on page load starts the boot while the user pastes, a visible tab re-pings every 10 min (under the sleep timer), a status pill explains a slow wake, and slow fetches / errors offer context plus retry. A paid instance is the real fix; a cron pinger would hide the problem instead.
Duplicated activation logic	`_hosted_levels_active()` (`youtube.py:54`) re-implements the same "key present AND flag enabled" conditions as `build_provider()` (`providers.py:346`). It must, because the proxy resolver runs per request inside a provider while the chain is built once at import.	Extract a single `level_is_active(name) -> bool` helper that both call. Real coupling, small blast radius — but it's exactly the kind of thing that silently diverges.
Hardcoded languages	`UrlInput.tsx` offers five (`en es hi de fr`). The backend accepts any code and Supadata supports more.	Return available caption languages from the API (`youtube-transcript-api`'s `list()` already fetches them) and populate the `<select>` from that.
`SITE_URL` **triplicated** FIXED	The Vercel domain *was* hardcoded in `layout.tsx`, `sitemap.ts` and `robots.ts`.	Fixed during the monorepo migration: it now lives once as `SITE_URL` in `@shrutilipi/shared` (`packages/shared/src/index.ts:46`) and all three files import it. This is the payoff the shared package was created for.
Stale doc claim FIXED	`operations.md` *said* "static export" while `next.config.js` had no `output: 'export'`.	Fixed: `operations.md` §2 now states it is a default Vercel build, not a static export — pick one and make the doc true, done.

GAP	WHY IT EXISTS	HOW I'D FIX IT
CORS allowlist drift FIXED	Adding the custom domain to the Vercel project changed the browser's `Origin`, but the API's allowlist was still pinned to the old Vercel URL via `render.yaml`. Preflight: `400 "Disallowed CORS origin"`. The simple `GET /api/wake`: 200 with *no* `access-control-allow-origin`, so the browser discarded every response — the page looked fine while no API call worked.	Fixed: the production origins are hardcoded in `main.py` (`PROD_ORIGINS`) and `FRONTEND_URL` is now additive-only, so a domain change can never lock the site out of its own API again. The comma-splitting bug is fixed too (split before adding), and both are pinned by `tests/test_cors.py` — 11 CORS tests, including one that asserts the production origins survive an unrelated env value.

PART B

Interview preparation

Backend and API heavy, as requested. Every question has: what the interviewer is actually probing, a model answer in your voice, the follow-ups to expect, and the trap to avoid.

16 How to use this

- **Read Part A out loud** — especially §02 (the constraint) and §08 (the chain algorithm). If you can narrate those two without looking, you can answer the first fourteen questions.
- **Answer from memory first**, then check the model answer. The gap between the two is your study list.
- **Say the "what they're probing" line to yourself** before reading the answer. Knowing the question behind the question is the difference between sounding prepared and sounding rehearsed.
- **The traps are the highest-value content.** Each one is a real way this project gets misunderstood — avoid them and you sound like someone who has operated the thing, not just written it.
- **Use §15 as your closer.** "Here's what I'd fix and why" is the most confident thing you can say about a small project, because you can't bluff a self-critique.

17 Question bank

Filter by topic and difficulty, then click a question to reveal the answer.

Showing 38 of 38 questions

Q1 Walk me through this project.

NARRATIVE EASY

WHAT THEY ARE REALLY PROBING

> The most common opener. They are testing whether you can describe your own work without burying the interesting part, and whether you actually understand your own codebase.

MODEL ANSWER

ShrutiLipi turns a YouTube URL into a plain-text transcript. Three entry points — a Next.js frontend on Vercel, a FastAPI backend on Render, and a stdlib-only Python CLI — over one backend. One endpoint, `GET /api/transcript?url=&lang=`, returning video ID, title, language, plain text and timestamped segments.

The interesting part is what happened when I deployed it. The obvious implementation, calling `youtube-transcript-api` directly, worked on my home connection and returned a 502 in production, because YouTube blocks datacenter IPs. That single failure is the entire reason the backend has an abstraction layer.

So the backend is a provider chain: three levels assembled from env at startup, which classifies each failure and reacts differently — not-found narrows the remaining providers, blocked-or-out-of-quota falls through, and video-unavailable stops immediately. Around that, a 24-hour in-memory cache, a typed error taxonomy mapping to stable detail codes, and shareable URLs where the address bar is rewritten to the canonical video ID so the result itself is the share link. About 2,190 lines, four Python dependencies, in a pnpm/Turborepo monorepo.

FOLLOW-UPS TO EXPECT

- Why the provider chain instead of one reliable provider?
- What's the tech stack and why?
- What would you add next?

TRAP TO AVOID

"Don't:" Do not open with the abstraction layer. Get the shape of the product on the table first, then earn the design detail. Burying the lead in sentence one sounds like you're apologising for the project.

Q2 What's the actual problem this solves? Who uses it?

NARRATIVE EASY

WHAT THEY ARE REALLY PROBING

> Whether you built something real or something that only makes sense to you. Also checking that you can be honest about scale and audience rather than inflating it.

MODEL ANSWER

The problem is small and specific: getting a clean, timestamped transcript out of a video is tedious. You either scrub through YouTube by hand, or you install `yt-dlp` plus a caption parser, or you paste into a site that asks for your email. ShrutiLipi is one paste and one button, with copy and download built in.

Honestly, the audience is small right now — mostly me, plus whoever finds the link. I built it to learn the shape of a deployed product, and the transcript quality is good enough that I use it myself. The honest version of "who uses it" is that I haven't instrumented it, so I genuinely don't know the traffic beyond a few test requests. If I were serious about adoption I'd add rate limiting, usage analytics and better handling of caption-less videos — which is exactly the v2 Whisper work.

FOLLOW-UPS TO EXPECT

- How would you find out whether people actually want this?
- What would you build first if you were serious about growth?

TRAP TO AVOID

"Don't:" Don't invent traction. Saying "I haven't instrumented it, so I don't know the numbers, and here's what I'd add to find out" is a far stronger answer than a vague claim about users.

Q3 What's the most interesting technical decision you made?

NARRATIVE MEDIUM

WHAT THEY ARE REALLY PROBING

> They want to see whether you can identify a decision with real trade-offs rather than just naming a technology. This is your chance to lead the conversation toward the provider chain.

MODEL ANSWER

The provider fallback chain, and specifically that it isn't a linear retry. A normal fallback chain calls A, then B, then C on any failure. That wastes money, because the failures aren't equivalent.

If a provider says "this video has no captions", that's a fact about the *video*, so calling another captions-only provider is a guaranteed miss that burns quota. If a provider is blocked or out of quota, that's a fact about *the provider's reach*, so a different provider on a different IP is genuinely likely to work. And if the video is private, nothing will ever work, so you should stop immediately rather than calling two paid APIs to be told the same thing.

So the chain classifies each failure and picks one of three reactions: narrow, fall through, or short-circuit. The narrowing policy lives as a class attribute on each provider — `handles_missing` — so a new provider declares its own capability and the chain's control flow doesn't change.

FOLLOW-UPS TO EXPECT

- How do you decide which reaction for a new error type?
- What if the chain grows to five providers?

TRAP TO AVOID

"Don't:" Do not call it the strategy pattern. The moment you name a textbook pattern, the interviewer starts grading you against the textbook instead of listening to your reasoning.

Q4 Why FastAPI instead of Flask or Django?

API & FASTAPI | EASY

WHAT THEY ARE REALLY PROBING

> Justifying a stack choice, ideally with trade-offs rather than feature checklists.

MODEL ANSWER

FastAPI was the right size for the problem. The whole backend is two routes, so I wasn't choosing a framework for features — I was choosing one that was cheap to run and cheap to extend.

Three reasons. First, typed query parameters: `lang: str = Query("en", description=...)` gives me validation and a self-documenting OpenAPI schema in two lines, where Flask would mean writing that validation by hand. Second, the Starlette base makes the CORS middleware a one-liner, and the preflight handling comes free. Third, it runs well on Render's free tier with plain uvicorn — the Dockerfile is `python:3.11-slim` plus `uvicorn main:app`, and the process count is the same as Flask's.

I'd have picked Flask for a single-route script. Django is a better answer than either of us if this needed an admin panel, ORM, or auth, and none of those are in scope for v1. I did deliberately skip Pydantic response models, though, because nothing is persisted and the models would only restate the response — that's a real cost of the choice, since the response shape isn't self-documenting in the OpenAPI schema.

FOLLOW-UPS TO EXPECT

- What would make you switch?
- How do you deploy it?

TRAP TO AVOID

"Don't:" Don't recite the FastAPI feature list. The strong signal is naming what you gave up: no response models, no built-in validation layer, no auth scaffolding.

Q5 Walk me through your API surface. Why is this a GET?

API & FASTAPI MEDIUM

WHAT THEY ARE REALLY PROBING

> HTTP semantics, idempotency and cacheability. The GET choice is deliberate and worth defending.

MODEL ANSWER

Three paths, two handlers. `GET /api/wake` returning `{"ok": true}` — the browser wake probe, deliberately not named after "health" because some adblock filter lists block health-style URLs client-side, and JS cannot tell that apart from a dead server — with `GET /health` as the ops alias on the same handler, and `GET /api/transcript?url=&lang=` doing the work.

It's a GET because the operation is a read: given the same URL and language you get the same transcript, so it's idempotent, safe, and cacheable. It has no side effects — the only mutation is populating an in-process cache, which is an implementation detail, not a state change the client can observe. That also means the CORS policy can be maximally tight: `allow_methods=["GET", "OPTIONS"]`, because there's no mutating endpoint to protect.

The real cost of GET is that query parameters end up in URLs, which means they end up in access logs, browser history and Referer headers. That's fine here because a YouTube URL isn't sensitive, but it's the reason I would not put anything secret in a query string. And it's why the cache key is built from the *resolved* video ID rather than the raw parameter, so five different URL shapes for the same video share one cache entry.

FOLLOW-UPS TO EXPECT

- What would a POST version look like?
- How does the browser cache interact with your cache?

TRAP TO AVOID

"Don't:" Don't say "GET because it's faster." The reason is semantics and safety, and it's also what lets the CORS policy be read-only.

Q6 How would you version this API?

API & FASTAPI | MEDIUM

WHAT THEY ARE REALLY PROBING

> Compatibility thinking, and whether you understand that a deployed public API is a contract you can't take back.

MODEL ANSWER

I'd version at the route: `/api/v1/transcript`. Not because the API is complex — it's one endpoint — but because a deployed endpoint on a public domain is a contract, and I can't take the shape back once someone's script depends on it.

The important discipline is that the `detail` codes are the real API, more than the response shape. The frontend switches on `no_captions`, `youtube_blocked` and `provider_unavailable`, and so does the CLI's exit-code logic. So adding a new failure mode means adding a new code, and *not* repurposing an existing one — turning `youtube_blocked` from a 502 into a 403 would break the CLI's `1`-versus-`2` decision without changing a single line of the CLI.

Right now there's no version prefix, which I'd fix before I had real consumers. Since the CLI in this repo is a client, versioning is already slightly overdue — if I break `plainText` I break my own tooling.

FOLLOW-UPS TO EXPECT

- How do you deprecate a field?
- What would you add for backwards compatibility?

TRAP TO AVOID

"Don't:" Do not say "it's a small project, I'd just change it." That is the answer that marks you as someone who hasn't deployed a public API.

Q7 **What happens if the user submits an empty string, or a URL with trailing junk?**

API & FASTAPI | MEDIUM

WHAT THEY ARE REALLY PROBING

> Input validation, and how much you trust the boundary. A good place to show you think about adversarial input rather than just happy paths.

MODEL ANSWER

Empty string: `extract_video_id()` has an explicit `if not url_or_id: return None` guard, which returns `None`, which `get_transcript` turns into `400 invalid_url`. The frontend also blocks it client-side with `disabled={loading || !url.trim()}`, so the user sees the disabled button and the request never happens — but the server doesn't trust that.

Trailing junk is accepted on purpose. There are four regex patterns tried in order: `watch?v=`, `youtu.be/`, `/shorts|embed|live|v/`, and a bare 11-character ID. Each uses `re.search` rather than `re.match`, so a share link like `https://youtu.be/HHUsHkYhkcM?t=43&feature=share` resolves to `HHUsHkYhkcM` without me having to enumerate tracking parameters. Anything genuinely unparseable falls through all four and becomes the same 400.

There is no ReDoS risk worth worrying about: the patterns are anchored on fixed literal prefixes and capture a bounded character class, with no nested quantifiers.

FOLLOW-UPS TO EXPECT

- What about a YouTube playlist URL?
- What if someone passes a 500-character string?

TRAP TO AVOID

"Don't:" Do not say you sanitise the input. The right instinct here was to normalise aggressively (extract 11 characters) and then use that canonical value as the cache key, rather than to validate everything to death.

Q8 How do you test this API?

API & FASTAPI | MEDIUM

WHAT THEY ARE REALLY PROBING

> A chance to show self-awareness: be exact about what has tests (the rate limiter) and what doesn't (the chain), then demonstrate you know exactly what you'd write next.

MODEL ANSWER

Honest answer, in two parts. The rate limiter has a real suite: 16 pytest cases in `apps/api/tests/test_ratelimit.py` covering the token-bucket maths with an injectable clock, client-key extraction, env parsing, and two `TestClient` proofs (the 429 shape with `Retry-After`, and cached hits consuming nothing). The provider chain has none — I hand-verified five behaviours during development — chain-order parsing, `handles_missing` eligibility, terminal short-circuiting, the SerpApi time-unit inference and the proxy skip — and `docs/operations.md` says those should be re-added as a test file if the project grows. So I know exactly which parts are unverified, and they happen to be the highest-value parts to verify.

What I'd write, in order. First, `test_build_provider`: table-driven, with `monkeypatch.setenv`, covering the three return branches, keyless local dev, the `SERPAPI_ENABLED`-defaults-false gotcha, and unknown names in `CHAIN_ORDER`. That's pure logic with no network. Second, `test_fallback_provider` with stub levels — I can assert narrow, fall-through, short-circuit and exhaustion without touching YouTube or Supadata at all, which is the whole point of the ABC. Third, `test_extract_video_id` for the four URL shapes, and `test_normalize_serpapi` for the seconds-versus-ms heuristic.

What I would *not* do is mock `httpx` for the hosted providers. Those tests would assert my own assumptions about a third party's response shape, which is the definition of a test that passes while the product is broken. The real integration test is a request against the known-good video `HHUsHkYhkcM`, which `operations.md` already documents as the smoke test.

FOLLOW-UPS TO EXPECT

- How would you test the error mapping?
- Would you use FastAPI's TestClient?

TRAP TO AVOID

"Don't:" Do not overstate or understate coverage. The limiter suite is real and worth claiming; the chain has none, and `operations.md:168` is written down. Being caught overstating coverage would cost you every other claim you make.

Q9 Explain how the fallback chain works.

PROVIDER CHAIN MEDIUM

WHAT THEY ARE REALLY PROBING

> The core of the project. If you can only explain one thing well, make it this one.

MODEL ANSWER

Three levels, ordered by an environment variable called `CHAIN_ORDER`, default `direct,supadata,serpapi`. Direct is `youtube-transcript-api` hitting YouTube itself; Supadata is a hosted service with AI transcription; SerpApi is a hosted captions-only service.

All three implement one abstract base class, `TranscriptProvider`, with a single method `get_transcript(video_id, lang)` returning a `TranscriptResult`. That shared shape is what makes them swappable — the chain doesn't know which provider is which.

`build_provider()` walks `CHAIN_ORDER`, skips any level whose key is missing or whose enabled-flag is false, and then has three returns: no levels at all gives a bare `CaptionProvider` with a warning, one level is returned unwrapped, and more than one gets wrapped in a `FallbackProvider`. It runs once at import time, so changing env needs a redeploy.

At request time `FallbackProvider` pops levels off a mutable list and classifies every failure into one of three reactions. That's the interesting part and it's the next question.

FOLLOW-UPS TO EXPECT

- What are the three reactions?
- How is the chain ordered?
- What happens with no keys?

TRAP TO AVOID

"Don't:" Do not describe it as "trying providers until one works." That framing erases the entire design and the interviewer will ask why you built a custom loop instead of a for-loop.

Q10 Why not just retry the same provider, or loop through all of them?

PROVIDER CHAIN MEDIUM

WHAT THEY ARE REALLY PROBING

> Whether you can articulate the cost of naive retries. This is the question that separates a designed fallback chain from a written one.

MODEL ANSWER

Because the three failure classes have genuinely different economics.

A block or a quota error is about *this* provider's reach, so retrying a different one is likely to work — that's the normal case where fallback is right.

A "no captions" answer is a fact about the video, so calling another captions-only provider is a guaranteed miss. Supadata's free tier is 100 requests a month. If a user hits a caption-less video and the chain burns all three providers, I've spent three requests to learn one thing, and SerpApi's quota too. That's why the chain *narrows*: once it knows captions don't exist, it drops providers that can't do better than captions.

And a private or deleted video is terminal — no provider can read it, so calling two paid APIs to be told the same thing twice is pure waste. That one short-circuits.

There's a second reason too, not just cost: correctness of the answer to the user. If the video is gone, the user should be told the video is gone, not "we couldn't fetch a transcript." Retrying obscures the real reason.

FOLLOW-UPS TO EXPECT

- How much does a failed request cost you?
- What is your retry policy?

TRAP TO AVOID

"Don't:" Do not lead with latency. Lead with the fact that different failures deserve different treatment, then mention cost as the second reason. Latency is a symptom; misclassification is the bug.

Q11 What is `handles_missing`, and why is it a class attribute rather than a method or a dict lookup?

PROVIDER CHAIN HARD

WHAT THEY ARE REALLY PROBING

> API design and where policy lives. This is a genuinely good design question and a chance to show you think about extensibility.

MODEL ANSWER

`handles_missing: bool = True` is a class attribute on `TranscriptProvider`. It declares: "if a video has no captions, is it still worth calling me?" Supadata overrides it to `True` because it has AI transcription. SerpApi overrides it to `False` because it is captions-only.

Class attribute rather than a method because the answer is a *static property of the provider*, known at construction time. A method would imply it can vary per request, which it can't — SerpApi doesn't gain AI in production. As a dict lookup in the chain it would have to be a hardcoded mapping in the chain's control flow, so every new provider would mean editing the chain, and forgetting that edit is a silent bug where you just quietly waste money.

The payoff is that the chain's narrowing is a single generic filter, `pending = [p for p in pending if p.handles_missing]`, and adding a `WhisperProvider` in v2 requires no change to the chain at all. The provider declares its own capability and the existing policy applies. That's the whole reason I put it on the interface rather than in the chain.

The honest weakness: it's a single boolean, so it can't express "handles missing but only in English" or "handles missing but slowly." If I had more providers that distinction would probably become a small enum or a capabilities set.

FOLLOW-UPS TO EXPECT

- What if a provider could handle missing only in some languages?
- How would you unit test this?

TRAP TO AVOID

"Don't:" Do not say it's there for the strategy pattern. Say exactly what it's for: a new provider declares its own capability and the chain's control flow is untouched.

Q12 How do you decide which provider to use?

PROVIDER CHAIN MEDIUM

WHAT THEY ARE REALLY PROBING

> Configuration design, and whether you understand the difference between "configurable at runtime" and "configurable at build time."

MODEL ANSWER

From environment variables, read once at import time by `build_provider()`.

`CHAIN_ORDER` is a comma-separated try-order, default `direct,supadata,serpapi`. Each level additionally needs its API key present and its `_ENABLED` flag truthy. So a level is included only if it's named in the order, has its key, and isn't switched off.

Two details worth naming. First, `env_truthy` has three states, not two: unset falls back to a per-provider default, which is why `DIRECT_ENABLED` defaults true (captions should work out of the box on a home IP) while `SERPAPI_ENABLED` defaults false (it costs money). Second, `_parse_chain_order()` silently drops unknown names and duplicates instead of raising — a typo in a Render env var degrades to a slightly different chain rather than a service that won't boot.

The import-time read is a deliberate trade: building the chain is cheap but pointless per request, and env is immutable at runtime anyway. The cost is real though — changing the chain needs a redeploy, which is why `docs/operations.md` says env edits need “Save, rebuild, and deploy” on Render.

FOLLOW-UPS TO EXPECT

- How would you support per-request provider selection?
- Why not a config file?
- How do you test different configurations?

TRAP TO AVOID

“Don't:” Do not say "environment variables, it's twelve-factor." The interesting content is the three-state boolean, the silent dropping, and the fact that config is read at import rather than per request.

Q13 Walk me through exactly what happens when Supadata is out of quota. PROVIDER CHAIN HARD

WHAT THEY ARE REALLY PROBING

> End-to-end error propagation across three layers: the provider, the chain, and the HTTP mapper. Tests whether you can trace a failure rather than describe the happy path.

MODEL ANSWER

Supadata returns HTTP 429, or 402 if it's out of credit. `_map_hosted_status()` checks the status first and raises `ProviderQuotaError` for 402 and 429, before the body is even looked at. If instead the status were 200 with an error string in the body, `_raise_hosted_error()` would catch it — it matches keywords like quota, credit, exhaust, rate limit, too many, billing and plan.

Either way, `FallbackProvider` catches it. Now the two classification checks: `_is_terminal()` is False, because quota isn't a video problem. `_looks_missing()` is also False, because `ProviderQuotaError` is a typed `TranscriptError` whose `detail` is `provider_unavailable`, not `no_captions`. So the pending list is left untouched and the chain falls through to the next level.

SerpApi then serves the request. Nothing is logged as an error to the user — there's a `logger.warning` naming the failed level, and a `logger.info` naming who served it.

Now the important part: if SerpApi *also* fails, the chain is empty, so it raises the **last** error, which is SerpApi's. `_map_error()` sees a typed `TranscriptError` and returns 502 `provider_unavailable`, and the frontend turns that into “Transcript services are busy or out of quota — retry in a few minutes.”

FOLLOW-UPS TO EXPECT

- Why surface the last error rather than the first?
- What if the failure is intermittent?

TRAP TO AVOID

“Don't:” Do not stop at "it falls through to the next provider." The interesting part is the two checks that return False, and why a quota error must not be mistaken for a missing-captions error.

Q14 How do you map exceptions to HTTP status codes?

ERROR HANDLING | MEDIUM

WHAT THEY ARE REALLY PROBING

> API contract design, and whether your error responses are actually usable by a client.

MODEL ANSWER

There's one function, `_map_error()` in `main.py`, and it returns an `HTTPException` with a stable string `detail`. Six outcomes: `400 invalid_url`, `404 no_captions`, `404 video_unavailable`, `502 youtube_blocked`, `502 provider_unavailable`, `502 transcript_fetch_failed`.

The important design point is that the `detail` code is the contract, not the prose. The frontend's `friendlyError()` switches on that code and never on the status, and the CLI derives its exit code from it. So the code is what both clients depend on, and it means adding a failure mode is additive — a new code plus a new switch case — rather than a breaking change.

It works on two layers. Our own code raises typed exceptions carrying a `detail` class attribute, which is checked first and short-circuits. Everything else — exceptions from the unwrapped third-party library — gets classified by class name against three sets, with substring fallbacks on the message as a hedge against upstream renaming a class.

FOLLOW-UPS TO EXPECT

- Should error messages include detail?
- How would you add a new error code?
- What about validation errors?

TRAP TO AVOID

"Don't:" Do not return raw exception text to the client. It leaks internals — which provider failed, whether the API key is valid — and it's useless to the user anyway.

Q15 Why 502 for a blocked YouTube, and not 500?

ERROR HANDLING | MEDIUM

WHAT THEY ARE REALLY PROBING

> Whether you understand HTTP status semantics or just know the numbers. Also a chance to show you think about monitoring and retry policies.

MODEL ANSWER

Because ShrutiLipi is up and healthy — the upstream it depends on refused to serve the request. 502 Bad Gateway is the honest code for that, and it's useful: it tells monitoring that this is an external dependency problem, and it tells a client that a retry might work because the situation is likely transient.

500 would say “we have a bug.” That's untrue, and it's unactionable — a 500 alert would send someone hunting through application code when the actual cause is a block on someone else's network.

The 404s are equally honest in the other direction. “No captions” and “video unavailable” are facts about the resource: the thing you asked for genuinely doesn't exist, and retrying won't change that. 404 is cacheable and definitive, which is exactly right.

And 400 for a malformed URL is a client error the user can fix by pasting a different link. The one thing I deliberately avoided is mapping anything to 200 with an error body, because then every client has to inspect the payload and the CLI's exit codes get meaningless.

FOLLOW-UPS TO EXPECT

- What would make you use 429?
- What about 503?

TRAP TO AVOID

“Don't:” Do not treat this as a trivia question about status codes. The reasoning — that the code should tell the client whether retrying helps — is the whole answer.

Q16 What's the trickiest bug you've hit in error handling?

ERROR HANDLING HARD

WHAT THEY ARE REALLY PROBING

> Your best chance to show real debugging. Look for a problem with a root cause you understood rather than a stack trace you stared at.

MODEL ANSWER

`CouldNotRetrieveTranscript` from `youtube-transcript-api`. It's a catch-all in the library: it wraps IP blocks, consent-cookie failures, and genuinely-missing transcripts, and its class name is in none of my name sets. So the classification had to fall back to reading the message string — and the two places that do this read it in different orders, deliberately.

In `_looks_missing()`, which decides whether to narrow the chain, the **block** check runs first. If the message says "blocking requests" or contains both "ip" and "block", it returns False for missing. That ordering matters: if an IP block had been classified as missing, the chain would have narrowed to `handles_missing=True` levels, skipped SerpApi, and returned 404 "no captions" — a completely wrong answer for a completely temporary problem.

In `_map_error()` it's the mirror image: `"could not retrieve"` is a missing signal, so the function checks for the IP-block phrasing first and returns 502 `youtube_blocked` rather than falling through to 404.

So both do the specific check before the generic one. It's not elegant and it's string matching, which I'd normally avoid — but the alternative is guessing wrong about which of two very different problems happened, and a wrong guess here means both a bad error message and a wasted paid request. The real fix would be to wrap the library's exceptions in our typed hierarchy at the call site, which I haven't done; that would remove the string sniffing entirely.

FOLLOW-UPS TO EXPECT

- How would you make this more robust?
- How did you test it?
- What if the library changes its message?

TRAP TO AVOID

"Don't:" Do not present string matching as good practice. Present it as a deliberate, bounded hack with a known cost, and immediately name the real fix: wrap the third-party exceptions at the boundary.

Q17 Why do quota, auth and network errors share one detail code?

ERROR HANDLING HARD

WHAT THEY ARE REALLY PROBING

> Information leakage and API surface design. A strong candidate question for testing whether you think about what a client can infer.

MODEL ANSWER

Three reasons, and the first is the one that matters most.

The user's next action is identical for all three. The copy is the same in all three cases: “Transcript services are busy or out of quota — retry in a few minutes.” If I sent distinct codes I'd either be showing three different messages for the same advice, or inventing distinctions the user can't act on.

It would leak architecture. Telling a client “your provider's API key was rejected” versus “we're being IP-blocked” reveals which vendor is in use, whether the key is configured, and whether the block is IP-based. That's reconnaissance. The exception classes still carry that distinction internally, so my logs and debugging aren't degraded — only the wire format is.

It bounds the contract. Six detail codes instead of eight, and the frontend switch is short enough to read in one screen. If the vendored error wording changes and my keyword classification misfires, a quota error silently becomes a network error — and because they collapse to the same code, nothing visible breaks. That's a small blast radius by design, and it's the same reason I collapse the three hosted status codes the same way.

FOLLOW-UPS TO EXPECT

- Would you ever split them?
- What about logging — do you lose the distinction?

TRAP TO AVOID

“Don't:” Do not say "to keep the code simple." The real answer is that the three cases are indistinguishable to the user and distinguishable to an attacker.

Q18 Tell me about the caching.

CACHING & PERF | MEDIUM

WHAT THEY ARE REALLY PROBING

> Whether you can reason about cache keys, TTLs and invalidation, and whether you know your cache's limits.

MODEL ANSWER

It's a module-level dict, `_CACHE`, mapping `"videoId:lang"` to a tuple of stored-at-timestamp and payload, with a 24-hour TTL checked inline. That's the entire cache.

Three decisions worth defending. The key is the **resolved** video ID plus the language, not the raw query parameter, so five different URL shapes for the same video share one entry, and `en` and `hi` never collide. The TTL is 24 hours because captions for a given video effectively don't change, and because the free tier is 100 requests a month — this is the main reason the quota survives. And writes happen **only on success**, so a transient provider outage can't be cached as a failure. That last one is the important one: a poisoned cache entry would turn a thirty-second outage into a twenty-four-hour one.

Its limits, all of which are on my fix list. There's no eviction, so the dict grows unbounded and a long-lived process leaks memory slowly. It's process-local, so it's lost on restart and not shared — fine on Render's free single instance, but the day you run two workers each has its own cache and you pay twice for popular videos. And there's no single-flight, which is the concurrency problem I'll come to.

FOLLOW-UPS TO EXPECT

- Why 24 hours?
- How would you invalidate?
- What if captions change?

TRAP TO AVOID

"Don't:" Do not claim the cache is production-grade. Naming three concrete limitations unprompted is far more convincing than pretending a dict with no eviction is finished work.

Q19 What happens under concurrent load?

CACHING & PERF HARD

WHAT THEY ARE REALLY PROBING

> Concurrency reasoning. This is where most candidates stop thinking, and where the real weaknesses are.

MODEL ANSWER

Two problems, and only one of them is a race in the classic sense.

First, the **thundering herd**. If twenty users request the same uncached video at once, all twenty miss the cache and all twenty call Supadata. With a 100-requests-per-month free tier, one popular video can burn twenty requests in a second. The dict itself isn't the problem — under the GIL, individual dict operations are atomic enough that nothing corrupts — the problem is that there's no coordination between the check and the fill. The fix is per-key single-flight: a lock or an in-flight map keyed on `videoId:lang`, so the first request populates the cache and the rest await its result. A short negative cache for misses would help too, for the same reason.

Second, and more fundamental: the handlers are `def`, not `async def`. FastAPI runs them in a thread pool, so they don't block the event loop, but they're also synchronous blocking I/O — the Supadata call can sit for 25 seconds while holding a thread pool worker. Under enough concurrency that pool is the ceiling, and the answer is `httpx.AsyncClient` with `async def` handlers so one process can hold far more concurrent requests. I chose sync because the whole app is two routes and a chain of at most three calls, and sync `httpx` is simpler to reason about — but that's a decision that has an expiry date.

FOLLOW-UPS TO EXPECT

- How would you add single-flight?
- What is the thread pool size?
- Would you switch to async?

TRAP TO AVOID

"Don't:" Do not claim GIL safety means the cache is fine. Atomic dict writes prevent corruption; they do nothing about twenty simultaneous network calls.

Q20 How would you scale this?

CACHING & PERF HARD

WHAT THEY ARE REALLY PROBING

> System design under load. They want to see you identify bottlenecks in order and know what each fix costs.

MODEL ANSWER

In order of how much they actually matter here.

The provider quota is the hard ceiling, not CPU or memory. Supadata's free tier is 100 requests a month. Everything else is about not wasting them: the cache, single-flight, and a short negative cache for caption-less videos. If I needed more throughput, that's the constraint to attack first — self-hosting Whisper, or paying for a higher tier.

Rate limiting is second. A per-IP token bucket on `/api/transcript`, plus a `429` case in `friendlyError`. Right now the public API will happily let one client spend the whole monthly quota in an afternoon, which is a real availability problem for everyone else.

Async I/O is third, as I said — convert the providers to `httpx.AsyncClient` and the handlers to `async def`, so the process isn't limited by the thread pool while a 25-second upstream call is in flight.

The cache moves to Redis fourth, and only once I'm on more than one instance, because a process-local dict is actively harmful there — duplicated upstream calls and no shared warm state.

What I would *not* do first is add a database. Nothing here is user-owned or queryable; transcripts are re-derivable. A users table only appears when I add accounts, and that's a product decision, not a scaling one.

FOLLOW-UPS TO EXPECT

- Where would you add a queue?
- What about a CDN?
- How do you monitor this?

TRAP TO AVOID

"Don't:" Do not lead with "add a load balancer." The impressive answer identifies the actual bottleneck — a third-party free tier — and works outward from there.

Q21 Why a dataclass for `TranscriptResult`? What else could you have used?

PYTHON MEDIUM

WHAT THEY ARE REALLY PROBING

> Python data modelling. A chance to show you know the options and can justify a choice rather than default to the first thing that compiles.

MODEL ANSWER

A dataclass because it's a plain value object with four fields, one of which is optional, and it needs a derived property. That's exactly the dataclass use case: `__init__`, `__repr__` and `__eq__` for free, no inheritance, and the type hints stay in one place.

The specific detail I'm happy about is that `plain_text` is a **property**, not a stored field. It's recomputed from `segments` every time. That means the flat text the user copies can never drift out of sync with the segments they're looking at — there's no way to have one updated without the other, because there isn't a second copy.

The alternatives. A `TypedDict` would work if I only ever built these from dicts, but I need the property and a constructor, so it's a downgrade. A Pydantic model would give me validation and schema generation, which is genuinely useful — and it's the right call the moment I start validating untrusted provider responses instead of trusting them. I skipped it because in v1 the only construction site is code I wrote, so validation would be validating my own literals. A plain class with `__slots__` would save memory, which I don't need at this scale.

FOLLOW-UPS TO EXPECT

- How would Pydantic change this?
- What about immutability?
- Do you validate provider responses?

TRAP TO AVOID

"Don't:" Do not say a dataclass is "basically the same as a class with less code." The interesting part is the property, and being honest that Pydantic would be the better call once provider responses need validating.

Q22 You catch bare `Exception` in the chain. Isn't that bad practice?

PYTHON HARD

WHAT THEY ARE REALLY PROBING

> Defensive coding and whether you can justify a broad catch. A good question, because the naive answer is wrong.

MODEL ANSWER

In the chain loop it's necessary, and the reason is that the boundary is a third-party interface I don't control.

`FallbackProvider` iterates levels behind an abstract base class, so a level can raise anything: our typed errors, `youtube-transcript-api`'s twenty-odd exception classes, `httpx` transport errors, a `KeyError` from an unexpected response shape. If I caught a narrow set, any exception outside it would escape the loop, and the user would get an unhandled 500 instead of a fallback or a clean 404. So the catch-all is what makes the chain actually reliable.

Three things keep it defensible. The caught exception isn't discarded — it becomes `last_error` and is logged with the level name, class name and message before anything else happens. The chain is the outermost boundary, so catching here rather than inside each provider is the correct altitude: each provider can fail in ways only the chain can interpret. And a bare `return result` on empty segments is handled separately, so "succeeded with nothing" is treated as a miss rather than a success.

Inside the individual providers I'm much narrower — `httpx.HTTPError` around the calls, `(TypeError, ValueError)` around SerpApi's per-entry float coercion, and `except Exception: pass` in exactly one place, `get_title()`, which is a genuinely optional best-effort call where any failure should just mean no title.

FOLLOW-UPS TO EXPECT

- How would you test this?
- What about logging the traceback?
- Where else could you hide a bug?

TRAP TO AVOID

"Don't:" Do not apologise for the broad catch or claim it is required without saying what replaces it. The answer is: it's the outermost boundary, it logs rather than discards, and each provider is narrow where it can be.

Q23 Explain your `env_truthy` helper. Why not just compare to `"true"`?

PYTHON MEDIUM

WHAT THEY ARE REALLY PROBING

> Attention to configuration edge cases, which is a surprisingly good proxy for production experience.

MODEL ANSWER

Because the naive version has three bugs. `os.getenv("X") == "true"` treats unset as false, so a flag with a default of true needs to be written out in every environment or it silently flips. It misses `1`, `yes` and `on`, which people will inevitably type. And it doesn't strip whitespace, so a trailing space in a dashboard field — which is very easy to do by accident — makes the flag false.

My version has three states. `None` or empty means "use this provider's default." Anything in `0, false, no, off` means false. Everything else means true. So `DIRECT_ENABLED` defaults to true while `SERPAPI_ENABLED` defaults to false, and that asymmetry is expressed purely by which boolean is passed as the second argument at the call site.

It's a four-line function that exists because it's called in four places, and the alternative was four subtly different inline expressions that would drift. The real payoff is that the defaults are readable at the call site in `build_provider()` — you can see the whole policy in one screen. And `render.yaml` still sets `DIRECT_ENABLED=false` explicitly rather than relying on the default, because an explicit deployment config is easier to reason about than an implicit one.

FOLLOW-UPS TO EXPECT

- What if someone sets it to TRUE?
- How would you test this?

TRAP TO AVOID

"Don't:" Do not present it as over-engineering. The justification is the three states and the fact that four call sites share one policy.

Q24 How are you handling API keys?

SECURITY & CORS | EASY

WHAT THEY ARE REALLY PROBING

> Baseline security awareness. Keep the answer short and factual.

MODEL ANSWER

No key is in the repository. In `render.yaml` every secret is declared with `sync: false`, which tells Render not to read it from the file — it creates a placeholder row that I fill in through Dashboard → Environment → Secrets. So the values live only in Render's environment and in Vercel's, and `.gitignore` covers `.env` and `.env.local` as a second layer.

They're read with plain `os.getenv`, so they're in the process environment, which means they're not in the image and not in the source. The keys are only ever sent as HTTP headers to the provider — `x-api-key` for Supadata, a query parameter for SerpApi — and never returned to the client or logged. The proxy credentials get `quote(user, safe="")` before being interpolated into a proxy URL, so a password containing `@` or `:` can't corrupt the URL.

The honest weakness: the SerpApi key travels in a query string, because that's their API shape, so it'll appear in their access logs. That's a third-party constraint, not something I chose, and it's part of why SerpApi is off by default.

FOLLOW-UPS TO EXPECT

- What if a key leaks?
- How do you rotate?

TRAP TO AVOID

"Don't:" Do not say "they're in environment variables, so they're secure." Say what that does and does not buy you, and mention that env reads are import-time so a rotation needs a redeploy.

Q25 **Explain your CORS configuration.** SECURITY & CORS MEDIUM

WHAT THEY ARE REALLY PROBING

> A very common question and a common place for hand-waving. The good answer distinguishes CORS from access control.

MODEL ANSWER

The policy is an explicit allowlist: `FRONTEND_URL` (comma-separated for preview deployments) plus hard-coded production fallbacks, and a localhost-port regex (`^http://(localhost|127.0.0.1):d+$`) so local dev works on whatever port `next dev` lands on — a fixed port list broke the day a second project claimed 3000. Then `allow_methods=["GET", "OPTIONS"]` , `allow_headers=["*"]` , and `allow_credentials=False` .

Three reasoning points. First, an allowlist rather than a wildcard: `allow_origins=["*"]` would be perfectly fine for a public read-only API, and I'd probably use it, but an explicit list means a misconfigured or compromised frontend variable can't open the API to everything.

Second, and this is the bit that matters in an interview: **CORS is a browser mechanism, not access control.** It stops other *web pages* from reading my responses inside a user's browser. It does nothing against `curl` or my own CLI — which is exactly why the CLI works fine and why "CORS is my security" is a misunderstanding. There is no authentication in v1, so the API is genuinely public; CORS is purely about not letting arbitrary third-party sites embed it in their pages.

Third, `allow_credentials=False` is correct because there's no auth at all, and it also means `Access-Control-Allow-Origin: *` would be legal here — which is the exact check you have to make the day you do add cookies or tokens.

The operational trap: `FRONTEND_URL` has to match the Vercel domain *exactly*, scheme included. A mismatch produces a browser CORS error with nothing in the server logs, which cost me a deploy to learn.

FOLLOW-UPS TO EXPECT

- What happens when you add auth?
- Why include localhost?
- What about preflight?

TRAP TO AVOID

> **"Don't:"** Do not present CORS as a security feature. The strongest sentence is that it's a browser read-protection mechanism and does nothing against non-browser clients.

Q26 Is this API secure? What could an attacker do?

SECURITY & CORS HARD

WHAT THEY ARE REALLY PROBING

> Threat modelling under time pressure. They want honest risk assessment, not reassurance.

MODEL ANSWER

There is no authentication, so anything reachable on the internet is reachable. Concretely, the risks are:

Quota exhaustion. This is the real one. There *was* no rate limiting: one script could loop the endpoint and burn the monthly 100-request quota in an afternoon, making the app useless for everyone else. That's since fixed — a per-IP token bucket, a `429 rate_limited` path with `Retry-After`, and tests — but the quota itself is still the ceiling, so the fix beyond it is a paid tier or self-hosting.

Cost amplification. Each request can cost up to three upstream calls if the chain is fully active. Combined with the above, that's a direct financial exposure, which is why SerpApi is off by default and why the free tiers are what they are.

Abuse as a YouTube fetch proxy. The endpoint takes a URL parameter, so someone could point it at non-YouTube hosts. It doesn't actually do that — the four regexes only match YouTube shapes — but a stricter allowlist on the extracted ID's first character set is cheap defence in depth.

SSRF and injection are largely not applicable here, and it's worth saying so explicitly: there's no database, no shell, no template rendering, and no user-supplied filesystem paths. The one place user input flows into an outbound URL is the Supadata request, which reconstructs a canonical `watch?v=` URL from the extracted 11-character ID rather than passing the raw parameter through.

What I would not do is pretend CORS mitigates any of this. It's a browser mechanism; the CLI proves the API is open to non-browser clients.

FOLLOW-UPS TO EXPECT

- How would you add auth?
- What about the upstream providers' terms of service?
- Rate limit by what key?

TRAP TO AVOID

"Don't:" Do not lead with "yes, CORS is configured correctly." The honest answer starts with "there is no auth" and goes straight to the quota-exhaustion risk.

Q27 Walk me through the system design of this.

SCALING & DESIGN | MEDIUM

WHAT THEY ARE REALLY PROBING

> Whether you can describe a system you built without overclaiming. Keep it to what actually exists.

MODEL ANSWER

Browser and CLI are interchangeable clients. The browser talks to FastAPI over CORS with a single GET; the CLI talks to the same endpoint over plain `urllib` with zero dependencies. Both get the same five-field response.

The backend is a thin HTTP layer over one object. `main.py` does four things: CORS setup, a 24-hour TTL cache, the two routes, and one error mapper. Everything interesting is behind a single provider object built at import time, and the only entry point the rest of the app knows is `get_transcript(video_id, lang)`.

That object is a chain of at most three providers, each returning the same `TranscriptResult` dataclass. The chain is the only stateful, branching part of the system, and the branch condition is the failure classification.

Three cross-cutting decisions shape everything. Error handling is a typed taxonomy with stable string codes, because two separate clients depend on those codes. Configuration is env-driven and read at import, which makes local dev work with zero setup and production explicit. And the dependency list is deliberately tiny — four Python packages, and the hosted providers are called with plain `httpx` rather than vendor SDKs, so adding a provider costs no new dependency.

FOLLOW-UPS TO EXPECT

- Where would you add a queue?
- What's the data model?
- How do you handle the third-party failure?

TRAP TO AVOID

"Don't:" Do not describe a design you didn't build. There is no database, no queue, no user model, and no background worker — say so if asked rather than inventing them.

Q28 How would you add Whisper transcription for videos with no captions?

SCALING & DESIGN MEDIUM

WHAT THEY ARE REALLY PROBING

> Extensibility testing. This is the exact v2 the README promises, so there is a correct answer and it's short.

MODEL ANSWER

It's a new class implementing the existing ABC, and almost nothing else changes. `WhisperProvider(TranscriptProvider)` with `handles_missing = True`, because Whisper's whole purpose is handling missing captions. Implement `get_transcript(video_id, lang)` to return a `TranscriptResult` — download the audio, transcribe, chunk into segments with start and duration, and you're conforming to the interface.

Then it's configuration, not code: `WHISPER_API_KEY`, `WHISPER_ENABLED`, and append `whisper` to `CHAIN_ORDER`, plus the `_KNOWN_LEVELS` entry in `providers.py` and one branch in `build_provider()`. Redeploy, and it's live. No frontend change at all, because the frontend already handles `no_captions` gracefully and its copy already says “Whisper fallback coming in v2.”

Order matters, and it's a product decision more than a technical one: Whisper is expensive per minute of audio, so it should be last in the chain. But with `handles_missing=True` the narrowing already guarantees it's only reached after a cheaper captions-only provider has already said there are no captions — which is exactly the guardrail I wanted.

The real work isn't the adapter, it's audio handling: download, chunking, and cost control. At roughly a cent per audio minute, an unthrottled endpoint would be a genuinely different financial exposure from a 100-request caption API, and that's where I'd want rate limiting to exist first.

FOLLOW-UPS TO EXPECT

- What about the segments?
- Where would you run the audio?
- How would you handle long videos?

TRAP TO AVOID

“Don't:” Do not treat this as a coding exercise. The point is that the seam already exists, so the answer is "a new class plus three env vars and no frontend change."

Q29 If you had a thousand users tomorrow, what breaks first?

SCALING & DESIGN HARD

WHAT THEY ARE REALLY PROBING

> Prioritisation under hypothetical load. They want order and honesty, not a list of technologies.

MODEL ANSWER

In this order, and the first one isn't a technical problem.

The provider quota. A thousand users against 100 requests a month is off by four orders of magnitude, and it isn't fixable with engineering — you either pay for a higher tier or self-host. Before that, though, the cache absorbs most repeat traffic, so the honest first step is measuring cache hit rate, which I currently don't do.

Rate limiting has since shipped (per-IP token bucket, misses only), so the next cheapest protection is already in place — but the quota itself is still the ceiling, which is why it leads this list.

Render's free tier sleeps and cold-starts. A free instance idles down, so the first request after a quiet period pays a cold start — on the order of tens of seconds, which is a terrible first impression and would blow any sane timeout. Since then the frontend softens it: an `/api/wake` ping on page load starts the boot while you're still pasting a URL, a visible tab keeps the instance warm below the sleep timer, and a status pill plus wake button make a slow wake visible instead of mysterious. A paid instance is the real fix; a cron pinger would hide the problem rather than solve it.

Connection concurrency. The handlers are `def`, so FastAPI runs them in a thread pool, and each can hold a worker for up to 25 seconds on the Supadata timeout. Converting the providers to `httpx.AsyncClient` with `async def` handlers is the fix, and it's contained.

The cache, at scale-out. Process-local means N instances means N caches and N times the upstream calls for the same popular video. Redis at that point, or a single-flight lock per instance at least.

What does *not* break: there's no database, so no connection pool saturation; no user data, so no privacy or compliance surface; and the response is a few hundred kilobytes at most, so serving it is not the problem.

FOLLOW-UPS TO EXPECT

- What would you measure first?
- How would you know you're rate limiting too hard?

TRAP TO AVOID

"Don't:" Do not jump to "I'd add Redis and a load balancer." Identify the third-party quota as the ceiling and protect it with rate limiting before adding any infrastructure.

Q30 Why is the cache in memory? Why not Redis?

CACHING & PERF | HARD

WHAT THEY ARE REALLY PROBING

> Judging complexity against actual need. The interviewer is checking whether you add infrastructure by reflex.

MODEL ANSWER

Because Redis would be a second thing to run, secure, monitor and pay for, in exchange for solving a problem I don't have.

The requirements are: transcripts for a given video barely change, the backend is a single instance on a free tier, and a restart costs nothing but a cold cache. A dict satisfies all three. Redis earns its cost when you need the cache to be shared between instances, survive restarts, or be bounded by something other than process memory — and the first two of those arrive at the same time as "we're running more than one instance," which is a scale decision, not a caching decision.

What the dict genuinely can't do, and I want to be specific rather than hand-wave: it has no eviction, so a long-lived process grows without bound; it has no single-flight, so concurrent misses on the same cold key all call upstream; and it's lost on deploy, which is a cold-cache event rather than an outage.

So the honest framing is: the in-memory cache is the right call for the current deployment, and the migration trigger is running more than one instance. At that point it's not “let's add Redis” out of caution, it's “the cache is now wrong by construction” because a shared warm cache stops existing. I'd also add a size cap before that, since that's a two-line fix that removes the leak regardless of where the cache lives.

FOLLOW-UPS TO EXPECT

- What TTL would you use in Redis?
- How would you migrate without downtime?

TRAP TO AVOID

“Don't:” Do not justify in-memory caching as 'faster' or 'simpler' in the abstract. Justify it as 'correct for the current deployment, with a named trigger to change it.'

Q31 This project has no tests. Why?

TESTING & GAPS HARD

WHAT THEY ARE REALLY PROBING

> Self-awareness and honesty under direct challenge. How you handle a known weakness is more revealing than pretending it isn't there.

MODEL ANSWER

Honestly, it was a scope decision I made and then didn't revisit. My reasoning at the time was that v1 was a weekend-scale project with one endpoint, and the most valuable thing I could do was get it deployed and learn what actually breaks — which it did, and it was exactly the IP-block failure I'd never have found by testing locally.

That reasoning was reasonable for the first deploy and wrong as a steady state. The parts I'd want to test are precisely the parts that only fail in production: which provider serves a given video, how a quota error propagates, whether a caption-less video returns 404. None of those are testable without stubbing the providers, and stubbing them is what makes those tests fast and reliable anyway.

The cost of leaving it is real. The `SerpApi` seconds-versus-milliseconds heuristic is a genuine judgement call about a third party's undocumented behaviour, and it's exactly the kind of code that looks correct and quietly isn't. Same with the `CouldNotRetrieveTranscript` message ordering — I reasoned about it carefully once, in my head, and there's nothing keeping that reasoning true if someone edits the file later.

What I'd do now, in order: the table-driven `build_provider` tests, the `FallbackProvider` tests with stub levels, then `extract_video_id` for the four URL shapes, then a `pytest.mark.integration` test against the documented good video. A few hours, and it converts the parts I reasoned about into the parts I know.

FOLLOW-UPS TO EXPECT

- What would you test first?
- How do you test the chain?
- Would you write frontend tests?

TRAP TO AVOID

"Don't:" Do not say "I ran out of time." Say what the scope trade was, why it was right at the time and wrong now, and name the specific code you're least sure about.

Q32 What tests would you write first, and how?

TESTING & GAPS HARD

WHAT THEY ARE REALLY PROBING

> Whether you can turn self-criticism into a concrete plan. Saying "I'd add unit tests" without specifics is the wrong answer.

MODEL ANSWER

Two files, in this order, because they're pure logic and need no network.

`test_build_provider` — table-driven, using `monkeypatch.setenv` to set the env and then asserting on the returned chain. The cases: no keys at all gives a bare `CaptionProvider`; one key gives that provider unwrapped; two keys gives a `FallbackProvider` with the right levels in `CHAIN_ORDER` order; `SERPAPI_ENABLED` unset with a key present is ignored; unknown names and duplicates in `CHAIN_ORDER` are dropped; everything disabled triggers the empty-chain fallback. That last one is the documented gotcha, and it's the kind of thing a typo in a dashboard would silently change.

`test_fallback_provider` — with stub levels that implement the ABC and raise whatever you tell them to. Then assert the *reaction*, not just the outcome: a missing error leaves only `handles_missing` levels pending, a quota error leaves the list untouched, a terminal error is re-raised immediately, and exhaustion surfaces the *last* error. Those are exactly the four branches I reasoned about by hand, and they're the ones a future refactor could break silently. I'd also assert the call *order*, because a narrowing bug that calls one provider too many is a money bug.

Then `extract_video_id` against all four URL shapes, and `_normalize_serpapi` for the unit heuristic. What I would not unit test is the hosted providers with mocked `httpx` — those tests assert my assumptions about a third party, so they'd stay green while the integration broke. That belongs in one marked integration test against the known-good video.

FOLLOW-UPS TO EXPECT

- How do you test the time-unit heuristic?
- Do you use fixtures?
- What about the CLI exit codes?

TRAP TO AVOID

"Don't:" Do not say "I'd write unit tests with mocks for the providers." The distinguishing answer is that you test the chain with stub levels, not mocked HTTP, and explain why.

Q33 How do you debug a production issue here?

TESTING & GAPS | MEDIUM

WHAT THEY ARE REALLY PROBING

> Operational maturity. They want to know whether you've ever been on call for your own code.

MODEL ANSWER

Three concrete things, in order.

The log line that answers most questions. The chain logs `transcript served by %s video_id=%s segments=%d` on success and `level %s failed video_id=%s: %s: %s` on each failed level, with the exception class name. So for any failing request I can see the whole path — which levels were tried, in what order, and why each one gave up. That's the reason those two log statements exist; they're the observability, not decoration. `operations.md` even says the expected success line is `transcript served by SupadataProvider`.

A known-good video. `HHUsHkYhkcM` is documented with its expected shape — 969 `en` segments, title resolving via oEmbed. That gives me a fixed reference point: if it works, the chain and both providers are fine and the problem is the specific video; if it fails, the problem is upstream and the per-level log lines say which level.

Bisect the layers in order. `GET /health` first, to separate "the process is up" from "the transcript path is broken." Then `curl` the API directly — which also bypasses CORS entirely, and that's how I'd tell a frontend problem from a backend one. Then read the log line for which level served it.

The two failure modes I've actually hit: the Render IP block, which the per-level logs named immediately, and the `SERPAPI_ENABLED` gotcha, where a valid key did nothing because the flag defaulted to false — that one took longer because there was no error at all, the level was simply absent from the chain. That's a real gap in the logging: I'd add a startup line that prints the resolved chain, so "the level wasn't there" is never a hypothesis again.

FOLLOW-UPS TO EXPECT

- What metrics would you add?
- How do you know if it's the frontend or the backend?
- What about rate limiting your own debugging?

TRAP TO AVOID

"Don't:" Do not say "I'd add logging." Name the existing log lines, the known-good test vector, and the CORS-bypassing curl. Those are evidence you've debugged this in production.

Q34 Why a separate frontend and backend? Why not one app?

FRONTEND EASY

WHAT THEY ARE REALLY PROBING

> Basic architecture judgement, and a chance to show you considered the monolith option.

MODEL ANSWER

It's one repository with two deployable directories, so “separate” means separate deployments, not separate codebases. That split is driven by where each half has to live: the backend has to be reachable from a non-YouTube-blocked IP, which means a container on Render, while the frontend is a static bundle that Vercel serves from an edge network for free. One deployment target can't be both.

The cost is real and I'd name it: cross-origin requests need CORS configuration, and a mismatch on `FRONTEND_URL` fails in the browser with nothing in the server logs. The benefit is that each half scales and deploys independently, and neither has to be configured for the other's constraints.

If I were starting over with a different constraint — say, no cloud providers and everything on one box — I'd put the API in the same Next.js app under `app/api/` and ship one thing. The split here is a consequence of the deployment environment, not an architectural preference, and I'd rather say that than pretend monorepo-plus-two-hosts is inherently better.

FOLLOW-UPS TO EXPECT

- What if the frontend needed auth?
- How do you share types?
- What about a monorepo tool?

TRAP TO AVOID

“Don't:” Do not claim a monolith would be worse. The correct answer is that the deployment constraint forced the split, and you'd collapse it if that constraint went away.

Q35 The address bar holds the whole result state. Why?

FRONTEND MEDIUM

WHAT THEY ARE REALLY PROBING

> URL-as-state design. Worth asking because the implementation has a subtlety that took thought.

MODEL ANSWER

Because the result is fully determined by two query parameters, which means the URL *is* the state. There's nothing to serialise, and no result slug or ID to mint server-side.

Two consequences make it worth doing. Reload and share are the same code path — there's no separate share mechanism, and `ShareButtons.tsx` just builds the same query string the app already reads. And the page stays stateless enough that the back button behaves.

The implementation detail that took thought is the rewrite. After a successful fetch, `runFetch` calls `history.replaceState` with the **canonical resolved video ID**, not the raw input. So a `youtu.be` link with tracking parameters, a full watch URL, or a bare ID all converge on `/?url=dQw4w9WgXcQ&lang=en`. Users don't see that, but it means every shared link is normalised.

Two deliberate choices there: `replaceState` rather than `pushState`, so Back doesn't step through every search; and the whole call in a `try/catch`, because the History API can be unavailable and the only consequence should be that the shareable link doesn't update.

The one real limitation is that state transitions are coarse — Back returns you to the previous page, not the previous result within a session, because the URL only changes on success. If I wanted per-result history I'd `pushState` instead, at the cost of a Back button that gets tedious fast.

FOLLOW-UPS TO EXPECT

- Why not a route like /transcript/[id]?
- How do you handle the language changing?
- What if the URL is too long?

TRAP TO AVOID

"Don't:" Do not sell it as SEO. There's one indexable page; the query parameters don't create indexable URLs, and I wouldn't want them to.

Q36 Tell me about a bug you fixed that was hard to find.

BEHAVIOURAL MEDIUM

WHAT THEY ARE REALLY PROBING

> Debugging story. They want a real root cause, not a stack trace. This project has an unusually good answer.

MODEL ANSWER

The hardest one was `CouldNotRetrieveTranscript`. It kept producing 404 “no captions” for videos that I knew had captions, and only in production.

It's a catch-all exception in `youtube-transcript-api` that wraps IP blocks, consent-cookie failures *and* genuinely-missing transcripts, and its class name isn't in any of my name sets. So classification fell back to reading the message, and the message said “could not retrieve” — which my code treats as a missing-captions signal. From there it returned 404, the frontend showed “no captions found,” and the chain narrowed to `handles_missing` levels and skipped SerpApi.

Two things made it hard. It worked perfectly locally, because my home IP wasn't blocked, so the message never took the blocked path. And it was a *correct-looking* answer — a 404 with a clear message and a specific UI, not a crash. Nothing looked broken except the outcome.

What actually found it was reading the raw log line instead of the mapped response, and seeing the wrapped error. The fix was ordering: check the specific IP-block phrasing before the generic “could not retrieve” match, in both `_looks_missing()` and `_map_error()`. Two reordered conditions.

The lasting lesson is that the two functions that classify errors are the highest-risk code in the repo, they exist because I chose not to wrap the third-party exceptions, and there's no test pinning the ordering. So the thing I'd do next is exactly the test I said was missing.

FOLLOW-UPS TO EXPECT

- How long did it take?
- What would have caught it sooner?
- Did you change anything else?

TRAP TO AVOID

“Don't:” Do not describe a syntax error, a missing dependency, or anything resolved on first run. The interesting property of this bug is that it produced a confident wrong answer, not an obvious failure.

Q37 How do you decide what to build next?

BEHAVIOURAL EASY

WHAT THEY ARE REALLY PROBING

> Product judgement, and whether you can prioritise honestly on a project with no users.

MODEL ANSWER

Two different answers depending on whether the project has users.

With users, it's "what's the most common thing that fails for people?" In this project's terms, the only honest candidate is caption-less videos, because that's the one case where a user gets a dead end. That maps directly to the v2 Whisper work, which is why the README calls it out — not because Whisper is interesting, but because it's the gap in the user experience.

Without users, which is the actual situation here, I look for the thing that's cheap to fix and expensive to discover. That's how the missing tests and the missing startup log line both got onto my list: neither is visible in the product, and both cost real time the next time they bite. Then I look for anything that is a correctness risk rather than a missing feature — the SerpApi time-unit heuristic and the error-classification ordering both qualify, and both are cheap to pin down with a test.

What I try not to do is add capability that only the free tier of a third party makes possible. Right now that would mean Whisper, which is genuinely different in kind: a cent per audio minute instead of a hundred requests a month. That's a business decision about cost exposure, not a technical one, and it needs a rate limiter to exist first.

FOLLOW-UPS TO EXPECT

- What's on your list right now?
- How do you decide what's done?

TRAP TO AVOID

"Don't:" Do not answer with a feature roadmap. The credible answer starts from a real constraint of this project — the third-party free tier — and reasons about cost and risk.

Q38 Tell me about a technical decision you regret.

BEHAVIOURAL HARD

WHAT THEY ARE REALLY PROBING

> Self-awareness and intellectual honesty. A rehearsed non-answer here is a clear negative signal.

MODEL ANSWER

Two, and they're the same decision at different scopes.

Not wrapping the third-party exceptions. The library raises twenty-odd exception classes that carry no `detail` code, so the whole backend has a parallel classification mechanism built on class-name sets and message substring matching. It's the single riskiest design decision in the repo, and it exists because wrapping felt like boilerplate when I was getting a first deploy out.

The concrete cost showed up immediately: `CouldNotRetrieveTranscript` is a catch-all, so the code has to read its message, and the two functions that read it — `_looks_missing()` and `_map_error()` — have to keep their checks in a specific order for a reason that isn't obvious from reading them. I got that order wrong once and it produced confident, wrong 404s in production. The fix is a `try/except` at the call site in `fetch_caption_transcript()` that converts library exceptions into our typed hierarchy, at which point every string check after it can be deleted. It's maybe twenty lines. I should have written it first.

The cache, more broadly. No eviction, no single-flight, process-local. Each of those was a reasonable "not yet", and together they mean a public, unauthenticated endpoint can be made to waste a shared 100-request-per-month quota. I optimised for the simplest thing that worked and left the abuse case for later. Rate limiting is the first thing I'd add now, before any feature work.

What I took from it: the cost of a skipped boundary abstraction is paid later, at the worst possible moment, and the price compounds because the second and third occurrences of a pattern are the ones that should have triggered the refactor.

FOLLOW-UPS TO EXPECT

- What did you learn?
- Would you do the same again?
- What's the one thing you'd fix today?

TRAP TO AVOID

> **"Don't:"** Do not pick something cosmetic, and do not pick a decision you'd still defend. Pick the one with a real operational cost, name the cost in concrete terms, and state the fix you didn't make.

Filters and click-to-reveal are screen conveniences only. All 38 questions, with every answer, follow-up list and trap, are printed in full below in order, regardless of what was selected on screen.

18 Answer cheatsheet

The fifteen sentences worth memorising verbatim. If you only revise one page, revise this one.

#	SAY THIS
1	"The app is three entry points — a Next.js frontend, a FastAPI API and a stdlib CLI — over one backend, in about 2,190 lines with four Python dependencies."
2	"YouTube blocks datacenter IPs, so the obvious implementation works on my laptop and fails on Render. That single failure is why there's an abstraction layer at all."
3	"Transcript retrieval fails in structurally different ways, and each one deserves a different reaction — that's the whole design."

#	SAY THIS
4	“Three reactions: not-found narrows the remaining providers, blocked-or-out-of-quota falls through, and video-unavailable stops the chain immediately.”
5	“Narrowing is driven by a `handles_missing` class attribute, so the policy lives on the provider, not in the chain's control flow.”
6	“SerpApi has no AI, so once Supadata says ‘no captions’, calling it is a guaranteed miss that burns quota. The chain drops it.”
7	“The chain is built once at import time, so env changes need a redeploy. That's a deliberate trade for not doing work per request.”
8	“With no API keys the factory returns a bare direct provider rather than a one-element chain, so local dev exercises the real path with no indirection.”
9	“Config parsing drops unknown names and duplicates instead of raising, because a typo in a deploy-platform env var shouldn't take down production.”
10	“There are two error classification layers: typed exceptions with a `detail` for our own code, and class-name sets plus message sniffing for the unwrapped third-party exceptions.”
11	“`CouldNotRetrieveTranscript` wraps both IP blocks and genuinely missing captions, so the specific message check runs before the generic one — otherwise a block becomes a 404 and the chain narrows for no reason.”
12	“Quota, auth and network collapse to one detail code, because the user's next action is identical for all three, and separating them would leak which provider is in use.”
13	“The cache is a process-local dict with a 24-hour TTL, written only on success so a transient outage can't become a day-long one. No eviction, no shared cache, no single-flight — all three are on my fix list.”
14	“The current transcript result *is* the share link: after a successful fetch the address bar is rewritten to the canonical video ID, so share and reload are the same code path.”
15	“The biggest gap is no test suite. The chain is pure functions of chain-plus-outcome, so it's the easiest possible test target — five behaviours were hand-verified and I know exactly which ones.”

19 Your 2-minute and 10-minute stories

The 2-minute version — "walk me through this"

> *"ShrutiLipi turns a YouTube URL into a plain-text transcript. Three entry points: a Next.js frontend on Vercel, a FastAPI backend on Render, and a stdlib-only Python CLI. One endpoint,* `GET /api/transcript?url=&lang=`*, returning the video ID, title, language, plain text and timestamped segments.*
>
> *The interesting part is what happened when I deployed it. The obvious implementation — call* `youtube-transcript-api` *directly — worked perfectly on my home connection and returned a 502 in production, because YouTube blocks datacenter IP ranges. That is the entire reason the backend has an abstraction layer.*
>
> *So I built a provider chain. Three levels — direct captions, Supadata, SerpApi — assembled from environment variables at startup. The key design decision is that a fallback chain usually isn't a linear retry, because the failures aren't equivalent. If a provider says 'this video has no captions', calling another captions-only provider is a guaranteed miss that spends quota, so the chain narrows. If a provider is blocked or out of quota, that's about its reach rather than the video's content, so it falls through. And if the video is private or deleted, no provider will ever succeed, so the chain stops immediately and reports the real reason.*
>
> *That policy lives as a single class attribute,* `handles_missing`*, on each provider. SerpApi is captions-only so it sets it false; Supadata has AI transcription so it sets it true. The chain just filters on it.*
>
> *Around that: a 24-hour in-memory cache written only on success, a typed error taxonomy that maps to stable detail codes so the frontend can show specific copy, and shareable URLs where the address bar is rewritten to the canonical video ID so the result itself is the shareable link. I also wrote a CLI that shares the same quota and uses exit codes 0, 1 and 2 for scripting. About 2,190 lines, four Python dependencies, in a pnpm/Turborepo monorepo.*

The 10-minute version — architecture deep dive

TIME	COVER
0:00–1:00	What it is, the three entry points, the endpoint contract. **Do not** open with the problem yet — get the shape of the thing on the table first.
1:00–3:00	The production failure: datacenter IPs blocked, `502 youtube_blocked`, why the local version was a lie. This is your hook — most candidates never have a real "why" story.
3:00–6:00	The chain. Three levels, the ABC, `build_provider()`'s three return branches, then the algorithm walked line by line: narrow / fall through / short-circuit. Use the simulator in §09 live if the interviewer has a screen.
6:00–7:30	Error taxonomy. Both layers. The `CouldNotRetrieveTranscript` ambiguity is the best 90 seconds in the whole project — it shows you hit a real problem and reasoned about it.
7:30–8:30	Cache, CORS, env config. Mention the unbounded cache and the herd problem as known issues rather than waiting to be asked.
8:30–9:30	Frontend: the permalink design, the once-only bootstrap ref, progressive enhancement. Keep it short — this is a backend-weighted interview.
9:30–10:00	Close on the gaps. Name the missing test suite, the unbounded cache, the duplicated activation logic. Invite the follow-up question.

> **THE ONE THING TO GET RIGHT**
>
> Every answer in this document ultimately reduces to one claim: *the design exists because a specific production failure forced it, not because a tutorial said to build a strategy pattern.* If you sound like that, you sound like someone who has operated a system. If you sound like you're reciting a pattern, none of the rest matters.

20 Prep plan

IF YOU HAVE	DO THIS
10 minutes	Read §18 (the fifteen sentences) and §02. Say the 2-minute story from §19 out loud twice, timed.
1 hour	Add §08 (the chain algorithm) and §11 (errors). Rehearse the three reactions until they're automatic. Answer Q1–Q14 from memory.
3 hours	All of Part B. Write out the answers to the "Hard" questions in full sentences, then trim them to what you'd actually say in sixty seconds. Rehearse §15 as your closer.
A day	All of the above, then actually write the missing tests for `build_provider()` and `FallbackProvider`. After that you can say "I wrote them after this exercise" — a far better answer than the plan.

> **TWO QUESTIONS YOU WILL DEFINITELY BE ASKED**
>
> "What would you do differently?" → §15, lead with the tests.
> "What's the hardest bug you hit?" → the Render IP block, then the `CouldNotRetrieveTranscript` ambiguity.
> Both are real, both have root causes you understand, and neither is a "I forgot a semicolon" story.

Generated from `main @ 774d74d` plus the uncommitted pnpm-workspaces/Turborepo migration. Line references verified against the migrated working tree at the time of writing. Live: `shrutilipi.nayankunwar.com` · `shrutilipi-backend.onrender.com`. This document is a study aid, not documentation — the project's own `README.md`, `docs/operations.md`, `tools/README.md` and `tools/BATCH.md` are the sources of truth.